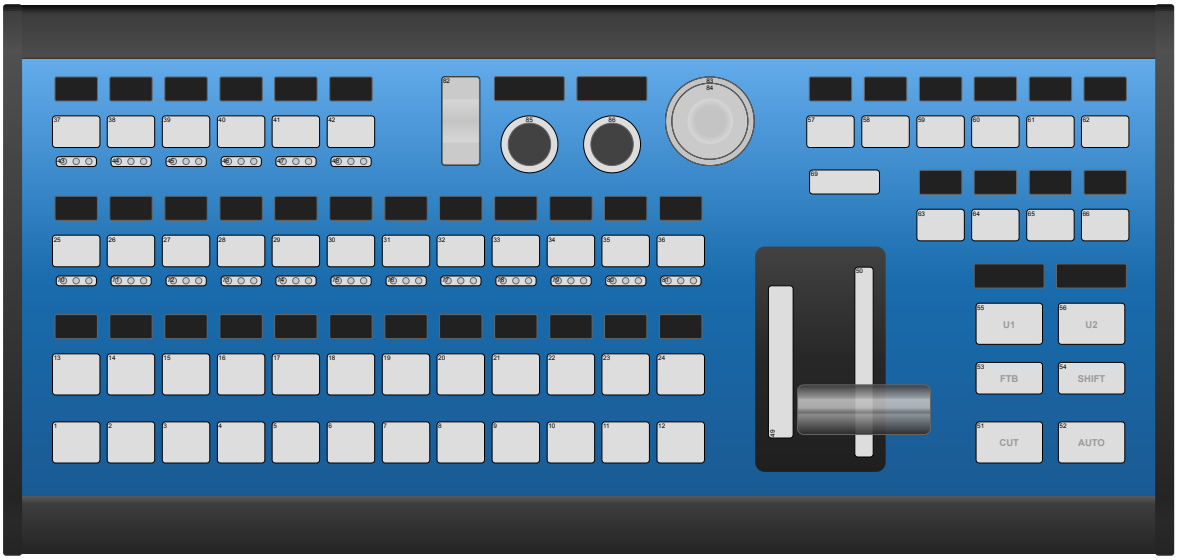
SKAARHOJ

Raw Panel Protocol

Protocol Reference

Integrating SKAARHOJ control panels with your own system:
commands, encodings, display content and topology.



Revision October 28, 2026 | Language: English

About this document

This document describes the Raw Panel Protocol, which lets your own software use SKAARHOJ control panels. Your software receives button presses, fader moves and encoder turns, and sends back LED colors and display content. Chapters 1–3 get you connected. Chapters 4 and 5 are the command reference, and the remaining chapters cover the details you need when you get further.

How to read the command reference

Every command is listed with its plain-text (ASCII) syntax, a short purpose, an ASCII example and the equivalent JSON. In the right margin:

`StateFeedback`	The `_support` flag a panel announces when it supports the command (page 38). Check it before you rely on the command.
Legacy	Kept for older panels and integrations. Do not use it in new work.
Since 2019	When the command was added. "≤2020" means it existed before 2021 without a known date.

Implementations add features at their own pace, so "supported" depends on the panel, not on a protocol version. Chapter 11 shows which implementation supports which feature.

Placeholders are written in `lower case`: `xx` is a hardware component number (HWC), `yy` a value. `<NL>` is a newline character (`\n`), which terminates every ASCII message.

Source of truth

The protocol is defined by the protobuf file `ibeam-rawpanel.proto` and by the ASCII converters in the open-source library `github.com/SKAARHOJ/rawpanel-lib`. All JSON examples in this document were generated with that library. If this document and the library ever disagree, the library is right.

Disclaimer. The Raw Panel Protocol is provided "as is" and may change without notice when SKAARHOJ finds it beneficial; see the changelog in Appendix B. We aim to keep the protocol backwards compatible, and panels built on the very first version still work, but compatibility of future versions cannot be guaranteed. SKAARHOJ assumes no liability for damages caused by third-party implementations of the protocol.

© 2026 SKAARHOJ ApS, Rosenkæret 11C, DK-2860 Søborg, Denmark • info@skaarhoj.com • www.skaarhoj.com. SKAARHOJ® is a registered trademark of SKAARHOJ ApS. Raw Panel, Blue Pill, Reactor and UniSketch are trademarks of SKAARHOJ ApS.

Contents

1 Introduction **6**

1.1 A first conversation . . . 6
1.2 Implementations and features . . . 7
1.3 Server mode and client mode . . . 7

2 Connecting to a Panel **8**

2.1 Finding panels . . . 8
2.2 Three encodings . . . 9
2.3 Protocol auto-detection . . . 10
2.4 Starting a session . . . 11
2.5 Keep-alive . . . 11
2.6 Several clients . . . 12
2.7 Hardware components and the map . . . 12

3 Command Index **13**

Inbound . . . 13
Outbound . . . 16

4 Inbound Commands **18**

4.1 LED and display feedback . . . 18
4.2 Session and panel information . . . 22
4.3 Panel settings . . . 24
4.4 Status, network and registers . . . 26
4.5 TouchUI . . . 28
4.6 Service and diagnostics . . . 30
4.7 Legacy . . . 32

5 Outbound Commands **33**

5.1 Events . . . 34
5.2 Availability map . . . 35
5.3 Panel information . . . 36
5.4 Topology . . . 39
5.5 Flow control . . . 40
5.6 Status and replies . . . 41

5.7 TouchUI . . . 45
5.8 Service . . . 45
5.9 Legacy (UniSketch only) . . . 46

6 LEDs and Indicators 47

6.1 Color palette . . . 47
6.2 Blink patterns . . . 48
6.3 VU meter thresholds . . . 48
6.4 Legacy jog wheel configuration . . . 49

7 Displays 50

7.1 Tile sizes . . . 50
7.2 Labels and values . . . 51
7.3 Text format . . . 51
7.4 Bitmaps . . . 55
7.5 Processors . . . 57

8 TouchUI 62

8.1 How it works . . . 62
8.2 Pages and layout . . . 62
8.3 Widget types . . . 63
8.4 Several clients . . . 63

9 Panel Topology 64

9.1 SVG background . . . 64
9.2 JSON components . . . 65
9.3 Topologies change . . . 66

10 Tools and Libraries 67

10.1 Raw Panel Explorer . . . 67
10.2 Raw Panel Dummies . . . 68
10.3 rawpanel-lib . . . 69
10.4 raw-panel-utils . . . 70
10.5 Raw Panel Bridge . . . 70

11 Implementation Support 71

A UniSketch Panels 75

A.1 Configuration . . . 75

A.2 Blue Pill Direct Mode 76
A.3 Client mode 76
A.4 Registers and change notifications 76

B Changelog **77**

1 Introduction

SKAARHOJ panels normally speak the protocols of the devices they control. With the Raw Panel Protocol, a panel does the opposite: it becomes a set of buttons, encoders, faders and displays that your system drives. The panel reports what the operator does, and your system decides what the LEDs and displays show. SKAARHOJ uses the same protocol internally to connect its own platforms, Blue Pill, Link IO and UniSketch (legacy).

- **TCP/IP**, port 9923 by default. Panels announce themselves with mDNS.
- **Event based.** The panel sends an event when a component is used; your system sends feedback whenever its state changes.
- **Self-describing.** A panel delivers its own layout (topology) as SVG and JSON, so you can support panels you have never seen.
- **Three encodings** on the same port: plain-text ASCII, JSON, and binary Protocol Buffers.

1.1 A first conversation

Connect to a panel with any TCP terminal (`nc 192.168.10.99 9923`) and type `list`. This is what a short session looks like in plain-text ASCII:

Session

→	`list`	
←	`_model=SK_RACKFUSIONLIVE`	
←	`_serial=435775`	
←	`_name=Studio A Fusion`	
→	`HWC#38=36`	turn on the LED of component 38
→	`HWCc#38=137`	…and make it yellow
→	`HWCt#38=|||Camera|1|CAM 1`	show a title and a text line in its display
←	`HWC#38=Down`	the operator presses the button
←	`HWC#38=Up`	…and releases it
←	`HWC#25=Abs:768`	a fader moves to about three quarters
←	`HWC#42=Enc:2`	an encoder turns two steps clockwise

Lines marked → go from your system to the panel (*inbound*). Lines marked ← come from the panel (*outbound*). Every hardware component has a number, its *HWC* id. Most of the protocol consists of events from HWCs and feedback to HWCs.

1.2 Implementations and features

The protocol is implemented by several products, each with its own feature set:

Implementation	Used by
`Blue Pill Platform`	SKAARHOJ panels on the Blue Pill platform, where the Hardware Manager component runs Raw Panel. Binary, ASCII and JSON.
`Link IO`	Standalone products such as Smart Tally, TallyBox, ETH-GPI and ETH-SDI, and the devices that used to run UniSketch. ASCII only, no JSON.
`External panels`	Drivers that make third-party hardware such as Stream Deck and X-keys available as Raw Panels. Built on the `rawpanel-server` Go library. Binary, ASCII and JSON.
`Raw Panel Dummies`	Emulates SKAARHOJ panels in a web browser.
`UniSketch`	The legacy SKAARHOJ platform. ASCII only, no JSON, with fewer features than the others.

There is no protocol version number to check. Instead, a panel announces optional features in its `_support` flags (page 38), and ignores commands it does not know. Earlier documents called UniSketch's ASCII protocol *V1* and everything newer *V2*; the mDNS field `rwp2.0` still reflects that.

1.3 Server mode and client mode

In **server mode** the panel is the TCP server and your system connects to it. This is how almost all integrations work today, and it is the only mode on Blue Pill panels. A panel in server mode accepts several clients at once, which allows redundant control systems.

Client mode is the original arrangement: the panel connects out to a server on your system, by default on port 9923. It is legacy but still supported. UniSketch panels offer it directly (Appendix A). Blue Pill panels get the same effect with **Raw Panel Bridge**, which connects to the panel locally and opens an outbound TCP or WebSocket connection to a remote system, for example a cloud service behind a firewall.

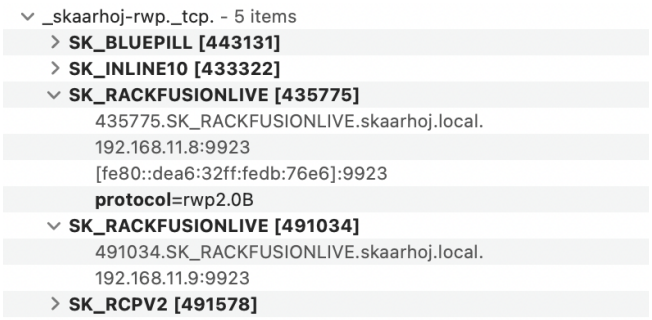
This document assumes server mode unless stated otherwise.

2 Connecting to a Panel

2.1 Finding panels

Panels announce themselves on the local network with mDNS (Bonjour/Zeroconf) as service `_skaarhoj-rwp._tcp`. The service name contains model and serial number, and the TXT record field `protocol` tells you which encoding to use:

protocol	Meaning
`rwp2.0B`	Binary (Blue Pill)
`rwp2.0A`	ASCII, with JSON
`rwp2.0A/B`	Auto-detects ASCII or binary per connection (see below)
`rwp1.0A`	ASCII only (Link IO products)
(none)	ASCII only (UniSketch)



Two Rack Fusion Live panels found with mDNS. Serial 435775 is a Blue Pill panel (`rwp2.0B`); serial 491034 announces no protocol and is a UniSketch panel.

Connect with TCP to the announced port, normally **9923**.

2.2 Three encodings

All three encodings carry the same messages. Pick the one that suits your platform. A panel's `_support` flags (page 38) tell you which ones it accepts.

Plain-text ASCII

Human-readable lines terminated by `<NL>`. This is the easiest way to start: you can type commands in a terminal and read the replies. It gets clumsy for display text with many parameters and for images, which must be base64 encoded and split over several lines. Plain-text ASCII is the only encoding UniSketch panels understand.

ASCII

```
HWC#38=36
```

JSON over ASCII

On the same ASCII connection most panels also accept JSON, one message per line with whitespace removed. The JSON is the protobuf message structure in JSON form, so it is as complete as the binary encoding while still being text. The first character of the line decides how it is read:

Line starts with	The panel reads it as
`{`	One *state* for one or more HWCs (an `HWCState` object): LEDs, colors, text, graphics, processors. Requires support flag `JSONFeedback`.
`[`	An array of complete inbound messages (`InboundMessage`): any command, flow message or state. Requires support flag `JSONNonInbound`.

ASCII

```
HWC#38=36
HWCc#38=137
```

JSON

```
{"HWCIDs":[38],"HWCMode":{"State":4,"Output":true},"HWCColor":{"
    ↪ ColorIndex":{"Index":9}}}
```

A state object can combine LED state, color and display content in one line, which several ASCII lines would otherwise be needed for. Commands use the array form:

ASCII

```
list
```

JSON

```
[{"Command":{"SendPanelInfo":true}}]
```

Panels that list `JSONNonOutbound` can also *reply* in JSON: send `JSONNonOutbound=1` and every outbound message arrives as a JSON array of `OutboundMessage` objects:

ASCII

```
HWC#35.4=Down
```

JSON

```
[{"Events":[{"HWCID":35,"Binary":{"Pressed":true,"Edge":4}}]}]
```

Binary (Protocol Buffers)

The native and most efficient encoding of Blue Pill panels. Each message is a protobuf-encoded `InboundMessage` (to the panel) or `OutboundMessage` (from the panel), preceded by a 4-byte **little-endian** length of the message:

Binary frame

```
[ length: uint32, little-endian ][ protobuf message: length bytes ]
```

Generate code for your language from `ibeam-rawpanel.proto` in rawpanel-lib, or use the Go library directly. Every JSON example in this document is the same structure you build in code. For example, the button event above, with a timestamp added, is this `OutboundMessage` in binary, length header first:

Binary bytes

```
[16 0 0 0  66 14 8 35 18 4 8 1 16 4 48 208 161 225 141 3]
```

> **Zero values.** Protobuf leaves out fields that hold their default value: `0`, `false` and empty strings. Treat a missing field as its default. A sub-message that is *present but empty* still counts, so `{"Command":{"SetSleepMode":{}}}` sets sleep mode 0.

Choosing

Encoding	Use it when
`ASCII`	You are exploring, scripting, or talking to Link IO or UniSketch panels.
`JSON`	You want structured messages without a protobuf toolchain, or need features that have no plain-text form (like the *Processors* feature).
`Binary`	You build a product. It is the fastest encoding and covers the whole protocol.

2.3 Protocol auto-detection

Panels announced as `rwp2.0A/B` start every connection in binary, also for the events they send. As soon as they receive four bytes that are not a plausible binary length header (more than 8 MiB, which any ASCII text is), they switch the connection to ASCII. Therefore send a command right after connecting. If your system supports both encodings, probe with a binary `ping`: a binary panel answers with a binary `ack` within 2 seconds, an ASCII panel does not. Send a newline afterwards to clear the ASCII panel's input buffer.

2.4 Starting a session

In server mode no handshake is required, but a good client does this after connecting:

1. Send `list` to learn model, serial, name and supported features (page 22).
2. Send `PanelTopology?` if you need to know the layout and display sizes (page 23).
3. Send `map` to learn which HWCs are active (page 22). Only send feedback to HWCs that are present in the map.
4. Send `HeartBeatTimer=3000` to have the panel ping you every 3 seconds (page 23). Answer each `ping` with `ack`.
5. Send the current feedback (LEDs, colors, displays) for all HWCs.
6. Process events as they arrive. Send new feedback whenever your system's state changes.

Session

```
→ list
← _model=SK_RACKFUSIONLIVE
← _serial=435775
← _version=v2.4.1
← _name=Studio A Fusion
← _platform=ibeam
← _panelType=BPI
← _support=ASCII,Binary,JSONFeedback,JSONNonInbound,...
→ map
← map=1:1                                   HWC 1 is present...
← map=2:2                                   ...and so on for every HWC
→ HeartBeatTimer=3000
← _heartBeatTimer=3000
← ping                                      3 seconds later
→ ack
```

2.5 Keep-alive

A TCP connection can fail without either side noticing until something is sent. With `HeartBeatTimer` enabled, the panel sends `ping` at the given interval and closes the connection if no message arrives from you within 2 seconds. Any message counts as an answer; `ack` is the natural choice. Some implementations send the first `ping` right after you enable the timer. Each client has its own timer. You can also send `ping` to the panel yourself; it answers `ack`.

2.6 Several clients

A panel in server mode accepts several simultaneous clients. Events go to every client, and feedback from any client is applied, so coordinate your clients or the last sender wins. A panel can be locked to specific IP addresses or limited to a number of clients. A rejected client receives an ASCII line before the panel closes the connection: `ErrorMsg=Sorry, max clients reached.` or, for an IP that is not allowed, `ErrorMsg=Sorry, it's a private party.`

2.7 Hardware components and the map

Every button, encoder, fader, joystick, display, GPI and GPO is a *hardware component* (HWC) with a number from 1 upwards. Events carry the HWC number, and feedback addresses one or more HWC numbers.

The `map` reply tells you which HWCs are active: `map=23:23` means HWC 23 is present, `map=23:0` means it is disabled. Panels send new map entries by themselves whenever this changes, for example when an optional module is attached, when a TouchUI page changes, or when the panel goes to sleep and wakes up. After a wake-up, re-send the feedback for HWCs that reappear. Details on page 35.

3 Command Index

All commands at a glance, in order of importance. Click a command to jump to its description. *Requires* names the `_support` flag a panel announces when it supports the command; commands without a flag cannot be detected in advance. Which implementation supports what is shown in chapter 11.

Inbound: your system → panel

Command	Purpose	Since	Requires	Page
LED AND DISPLAY FEEDBACK				
`HWC#xx=yy`	LED state: off, on, dimmed, blinking; GPO output	2017		18
`HWCc#xx=yy`	LED color, as a palette index or a 6-bit RGB value	2017		19
`HWCt#xx=string`	Display content as text: title, value, labels, scales	2017		19

`HWCg#xx=header:base64`
`HWCgGray#xx=header:base64`
`HWCgRGB#xx=header:base64`
	Display content as a bitmap: monochrome, grayscale or RGB	2017		20

`HWCs#xx=mode,color|mode,color|...`	Individual LEDs of an LED bar or tally strip	2026	`Segments`	20
`HWCx#xx=yy`	Extended output: LED bars, VU meters, motorized faders	2019		21
`HWCj#xx=mode|autoStop|stop|detents|target`	Complete configuration of a haptic jog wheel	2026	`JogConfig`	22
SESSION AND PANEL INFORMATION				
`list`	Ask the panel to describe itself	2017		22
`map`	Ask which hardware components are active	2017		22
`PanelTopology?`	Ask for the panel layout as SVG and JSON	2019		23
`state`	Ask for the current state of all components	2026	`StateFeedback`	23
`HeartBeatTimer=xx`	Enable keep-alive pings every xx milliseconds	2019		23
`ping`	Check the connection	2017		23
`ack` `nack`	Answer to a ping from the panel	2017		24
PANEL SETTINGS				

`Clear`
`ClearLEDs`
`ClearDisplays`
	Clear all LEDs and displays	2019		24

`PanelBrightness=x,y,z`	Brightness of LEDs, displays and screen	2020		24

Command	Purpose	Since	Requires	Page
DimmedGain=xx	Brightness of the dimmed state	≤2020		25
SleepTimer=xx SleepTimer?	Sleep after xx milliseconds without activity	2019		25
SleepMode=xx	How LEDs behave while the panel sleeps	2019		25
SleepScreenSaver=xx	What displays show while the panel sleeps	2019		25
WakeUp!	Wake the panel from sleep	2019		26
Reboot	Restart the panel	2019		26
JSONNonOutbound=x	Have the panel reply in JSON	2022	JSONNonOutbound	26
STATUS, NETWORK AND REGISTERS				
Connections?	List the connected clients	≤2020		26
RunTimeStats?	Ask for uptime and boot count	≤2020		27
NetworkConfig?	Ask for the network settings	2024	NetworkSettings	27
SetNetworkConfig={json}	Change and apply the network settings	2024	NetworkSettings	27
Registers?	List the panel's registers and their values	2022	Registers	27
MemX=n Flag#n=0/1 ShiftX=n StateX=n	Set a register	2022	Registers	28
TOUCHUI				
SetTouchUI={json}	Set or replace the widget configuration	2026	TouchUI	28
ClearTouchUI	Remove all widgets	2026	TouchUI	28
TouchUICapabilities? TouchUIConfig?	Ask for screen size and supported widget types	2026	TouchUI	29
SetTouchUIActivePage=n	Show another page	2026	TouchUI	29
HWCo#xx={json}	Overlay boxes on a widget, such as tracking frames on video	2026	TouchUI	29
HWCd#xx={json}	Change a widget's choices or range at runtime	2026	TouchUI	30
SERVICE AND DIAGNOSTICS				
HWCrawADCValues#xx=0/1	Stream raw analog readings of a component	2022	RawADCValues	30
PublishSystemStat=xx	Report system statistics every xx seconds	2021	System	30
LoadCPU=xx	Stress test xx CPU cores	2021	System	31
SimulateEnvironmentalHealth=xx	Test how your system reacts to a health warning	2022	EnvHealth	31
BurninProfile?	Ask for the burn-in test profile	2021	BurninProfile	31
CalibrationProfile? SetCalibrationProfile={json}	Ask for the calibration profiles	2022	Calibration	31
LEGACY				
ActivePanel=1	Activate a UniSketch panel in client mode	2017	Legacy	32
Webserver=x	Turn the UniSketch web server on or off	2020	Legacy	32

Command	Purpose	Since	Requires	Page
`encoderPressMode=xx`	How encoder presses are reported (UniSketch)	2019	Legacy	32
`(unknowncommand)`	Anything the panel does not understand	2019		32

Command	Purpose	Since	Requires	Page

Outbound: panel → your system

Command	Purpose	Since	Requires	Page
EVENTS				
`HWC#xx[.edge]=event`	An operator used a hardware component	2017		34
AVAILABILITY MAP				
`map=zz:xx`	Which hardware components are active	2017		35
PANEL INFORMATION				
`_model=string`	Model id	2017		36
`_serial=string`	Serial number	2017		36
`_version=string`				
`_name=string`	Friendly name of the panel	2021		36
`_panelType=string`	Kind of panel	2022		37
`_platform=string`	Software platform	2021		37
`_support=flag,flag,...`	Features the panel supports	2022		38
`_bluePillReady=0/1`	UniSketch: panel follows Blue Pill conventions	≤2020	Legacy	38
`_serverModeLockToIP=ip;ip;...`	Server mode: IP addresses the panel accepts	2019		39
`_serverModeMaxClients=n`				
TOPOLOGY				
`_panelTopology_svgbase=svg`	Background drawing of the panel	2019		39
`_panelTopology_HWC={json}`				
FLOW CONTROL				
`ping`	Heartbeat from the panel	2017		40
`ack`	The panel received your ping	2017		40
`nack`				
`BSY`	Panel busy: pause sending	2017	Legacy	40
`RDY`				
`list`	Client mode: the panel asks to be activated	2017	Legacy	41
STATUS AND REPLIES				
`_isSleeping=0/1`	The panel went to sleep or woke up	2019		41
`_sleepTimer=xx`	Sleep timeout in milliseconds	2019		41
`_heartBeatTimer=xx`	Heartbeat interval in milliseconds	2019		41
`DimmedGain=xx`				
`_connections=ip;ip;...`	Connected clients	≤2020		42

Command	Purpose	Since	Requires	Page

`_bootsCount=n`
`_totalUptimeMin=n`
`_sessionUptimeMin=n`
`_screenSaverOnMin=n`
	Runtime statistics	≤2020		42

`Msg=string` `ErrorMsg=string`	Information message	2019		42
`EnvironmentalHealth=xx`	Electrical noise around the panel	2022	`EnvHealth`	43
`SysStat=key:value:key:value:...`	System statistics	2021	`System`	43
`MemX=n Flag#n=0/1 ShiftX=n StateX=n`	Register values	2022	`Registers`	44

`HWCstate#xx[.edge]=value`
`HWCMode#xx=yy`
`HWCColor#xx=yy`
`HWCExtended#xx=yy`
	Current state of components	2026	`StateFeedback`	44

TOUCHUI				
`_touchUICapabilities={json}` `_touchUIConfig={json}`	Screen size and supported widgets	2026	`TouchUI`	45
`_touchUIActivePage=n`	The page the panel shows now	2026	`TouchUI`	45
SERVICE				
`_networkConfig={json}`	Network settings	2024	`NetworkSettings`	45

`_burninProfile={json}`
`_calibrationProfile={json}`
`_defaultCalibrationProfile={json}`
	Burn-in test profile	2021	`BurninProfile`	46

LEGACY (UNISKETCH ONLY)				
`HWCmsg#xx=CN:a,b,c,d,e`	Change notification	≤2020	Legacy	46

`_state:reg=xx _shift:reg=xx`
`_sleepMode=xx`
`_sleepScreenSaver=xx`
`PanelBrightness=x,y`
`Webserver=x`
	UniSketch register changes and setting echoes	2019	Legacy	46

4 Inbound Commands

From your system to the panel.

In all HWC commands, `xx` can be a comma-separated list of HWC numbers (not on UniSketch), for example `HWC#12,13,14=5`; the JSON field `HWCIDs` is always an array. JSON state examples show the single-line `{...}` form; commands show the `[...]` form (section 2.2). The binary encoding uses the same structures, wrapped in an `InboundMessage`.

4.1 LED and display feedback

HWC#xx=yy

Since 2017

LED state: off, on, dimmed, blinking; GPO output

Sets how the LED of a component lights up. `yy` is a bit field:

Bits	Meaning
`0–3`	State. `0` = off, `4` = on, `5` = dimmed. On and dimmed use the color set with `HWCc`, white by default. `1`, `2`, `3` = on in amber, red, green (legacy, use `HWCc` instead).
`4`	Blink flag for mono-color buttons (UniSketch only, legacy).
`5 (32)`	Output. Sets a binary output such as a GPO or relay tied to this HWC. Set it together with *on* and clear it otherwise.
`8–11`	Blink pattern 0–15. `1` blinks at about 4 Hz, `8` at 0.5 Hz. All patterns are shown on page 48.

In practice you send `0` (off), `36` (on, 32+4) and `5` (dimmed). Add 256 times the blink pattern to blink, for example `548` = on with output, blinking with pattern 2.

ASCII
```
HWC#38=36
```

JSON
```
{"HWCIDs":[38],"HWCMode":{"State":4,"Output":true}}
```

ASCII
```
HWC#12,13,14=5
```

JSON
```
{"HWCIDs":[12,13,14],"HWCMode":{"State":5}}
```

HWCc#xx=yy

Since 2017

LED color, as a palette index or a 6-bit RGB value

Sets the color a component shows when its state is *on* or *dimmed*. Setting a color does not turn the LED on; combine it with `HWC#xx=36` or `=5`.

Bits	Meaning
`7 (128)`	Enable. If clear, the panel uses its default color. Always set it.
`6 (64)`	Mode. Set: bits 0–5 are an RGB value `rrggbb` with 2 bits per channel. Clear: bits 0–5 are a palette index 0–18 (page 47).

Palette index 9 (yellow) is `128+9 = 137`. Pure red in RGB mode is `128+64+0b110000 = 240`. In JSON, `ColorRGB` takes 0–255 per channel and wins over `ColorIndex` if both are given.

ASCII
```
HWCc#38=137
```

JSON
```
{"HWCIDs":[38],"HWCColor":{"ColorIndex":{"Index":9}}}
```

ASCII
```
HWCc#38=240
```

JSON
```
{"HWCIDs":[38],"HWCColor":{"ColorRGB":{"Red":255}}}
```

HWCt#xx=string

Since 2017

Display content as text: title, value, labels, scales

Renders text on the display tile of a component. The panel takes care of layout and font sizes for the tile it renders on. `string` is a list of fields separated by `|`; empty fields take their default. The most used fields are:

Field order (start)
```
value|format|icons|title|isLabel|text1|text2|value2|...
```

The complete field list, rendered examples and the *label* versus *value* convention are in chapter 7. The ASCII string is limited to 63 characters of printable ASCII (32–127); for international characters use the `UniText` processor (page 57).

ASCII
```
HWCt#38=9999|2|40|Value
```

JSON
```
{"HWCIDs":[38],"HWCText":{"IntegerValue":9999,"Formatting":2,"
    ↪ ModifierIcon":5,"Title":"Value","SolidHeaderBar":true}}
```

ASCII
```
HWCt#38=|||Camera|1|CAM 1
```

JSON
```
{"HWCIDs":[38],"HWCText":{"Formatting":7,"Title":"Camera","
    ↪ Textline1":"CAM 1"}}
```

HWCg#xx=header:base64

Since 2017

Display content as a bitmap: monochrome, grayscale or RGB

HWCgGray#xx=header:base64 **HWCgRGB#xx=header:base64**

Sends a bitmap to a display tile. `HWCg` is monochrome (1 bit per pixel), `HWCgGray` is 4-bit grayscale and `HWCgRGB` is 16-bit color with the bits in blue-green-red order, not the usual RGB565 (section 7.4). Check the display type in the topology (`disp.type`) before sending gray or color images.

In ASCII the image data is base64 encoded and split over several lines of at most about 250 characters. The first line has a header `0/last,WxH` (and optionally an offset `,x,y`, which is not reliably supported); the following lines only carry their sequence number. In JSON and binary the whole image goes into one message. Byte layout, splitting and conversion tools are described in section 7.4.

ASCII

```
HWCg#34=0/0,16x8://+
    ↪ AAYABgAGAAYABgAH//w==
```

JSON

```
{"HWCIDs":[34],"HWCGfx":{"W":16,"H":8,"ImageData":"//+
    ↪ AAYABgAGAAYABgAH//w=="}}
```

`ImageType` is `0` (mono, omitted), `1` (RGB) or `2` (gray). The *processors* (section 7.5) can convert PNG, JPEG, GIF and WebP images for you.

HWCs#xx=mode,color|mode,color|...

Segments

Since 2026

Individual LEDs of an LED bar or tally strip

Sets every LED of a multi-LED component separately. Components that support it have `ext:"steps"` in the topology and one `sub` element per LED. Each entry is `mode,color` with the same values as `HWC` and `HWCc`; leave out `,color` for the default color. Entries follow the LED order (`_idx` 1..n); LEDs without an entry are off.

While set, segments replace `HWC`/`HWCc` for the component. Send `HWCs#xx=` with no value to go back to them. `HWCx` types 1, 3 and 4 take precedence over segments. Panels that do not support segments ignore the command, so also send a sensible `HWC`/`HWCc` state as a fallback. Supported if the panel reports `Segments`.

ASCII

```
HWCs#30=4,132|4,132|772,143|5,130
```

JSON

```
{"HWCIDs":[30],"HWCSegments":{"Segments":[
  {"HWCMode":{"State":4},"HWCColor":{"ColorIndex":{"Index":4}}},
  {"HWCMode":{"State":4},"HWCColor":{"ColorIndex":{"Index":4}}},
  {"HWCMode":{"State":4,"BlinkPattern":3},"HWCColor":{"ColorIndex":{"Index":15}}},
  {"HWCMode":{"State":5},"HWCColor":{"ColorIndex":{"Index":2}}}]}}
```

Two red LEDs, one blinking green, one dimmed white, and the rest off.

HWCx#xx=yy

Since 2019

Extended output: LED bars, VU meters, motorized faders

Drives components that show a value rather than a state. `yy` = *type* × 4096 + *value*:

Type	Value
`0 None`	Turns extended output off; the LEDs follow `HWC`/`HWCc` again.
`1 Strength`	0–1000, shown on an LED bar. On UniSketch it also holds a motorized fader in place (legacy; use type 5).
`3 Steps`	0 = all LEDs off, n = the nth LED on, more than the number of LEDs = full bar dimmed.
`4 VU meter`	0–1000 audio level with meter colors (thresholds on page 48).
`5 Fader`	0–1000. Moves a motorized fader to the position once.
`6`	Reserved, not used.
`7 Jog wheel`	Legacy bit-packed jog configuration. Use `HWCj` on panels that support it.

Bits 10–11 of the value are reserved for the individual types. The topology tells you which components accept extended values (`ext`, page 66).

ASCII
```
HWCx#21=16884
```

JSON
```
{"HWCIDs":[21],"HWCExtended":{"Interpretation":4,"Value":500}}
```

ASCII
```
HWCx#21=21230
```

JSON
```
{"HWCIDs":[21],"HWCExtended":{"Interpretation":5,"Value":750}}
```

The first example shows a VU level of 500, the second moves a fader to 750.

HWCj#xx=mode|autoStop|stop|detents|target

JogConfig

Complete configuration of a haptic jog wheel

Since 2026

Configures a motorized jog wheel in one message. Every message replaces the whole configuration, and the panel only reconfigures fields that changed, so it is safe to resend the configuration with every position update. Supported if the panel reports `JogConfig`.

Field	Meaning
`mode`	`0` detents light, `1` detents with assist, `2` detents, `3/4` detent mixes, `5` free, `6` free with assist, `7` stop, `8` stop full, `9` shuttle, `10` volume knob, `11` volume knob with end bumps.
`autoStop`	Shuttle only: `1` = stop the shuttle automatically.
`stop`	Shuttle only: `1` = allow stopping and holding the shuttle.
`detents`	Modes 0–6: detents per revolution, 2–255. `0` keeps the default (about 25).
`target`	Modes 10–11: position 0–1000 to move to. Leave empty to not move the motor.

ASCII

```
HWCj#40=10|0|0|0|500
```

JSON

```
{"HWCIDs":[40],"HWCJog":{"Mode":10,"TargetPosition":{"Value"
    ↪ :500}}}
```

4.2 Session and panel information

list

Since 2017

Ask the panel to describe itself

The panel answers with its model, serial number, software version and, except on UniSketch, name, platform, panel type and supported features. Each value is a separate line; see section 5.3 for all of them. Send `list` first after connecting.

ASCII

```
list
```

JSON

```
[{"Command":{"SendPanelInfo":true}}]
```

map

Since 2017

Ask which hardware components are active

The panel answers with one `map=zz:xx` line per HWC (page 35). Some panels send the map by themselves when you connect, but do not rely on that. Changes are always sent automatically.

ASCII

```
map
```

JSON

```
[{"Command":{"ReportHWCavailability":true}}]
```

PanelTopology?

Since 2019

Ask for the panel layout as SVG and JSON

The panel answers with `_panelTopology_svgbase` and `_panelTopology_HWC`: a background drawing and a description of every component, including display sizes. See chapter 9.

ASCII
```
PanelTopology?
```

JSON
```
[{"Command":{"SendPanelTopology":true}}]
```

state

StateFeedback

Since 2026

Ask for the current state of all components

The panel answers with the current physical state of its components (button pressed, fader position, joystick deflection) and the feedback they currently show. Useful right after connecting, or when other clients may have changed the panel. Panels without support ignore the command. Supported if the panel reports `StateFeedback`; today that is the Link IO products (chapter 11). The reply format is described on page 44.

ASCII
```
state
```

JSON
```
[{"Command":{"SendStateFeedback":true}}]
```

HeartBeatTimer=xx

Since 2019

Enable keep-alive pings every xx milliseconds

The panel sends `ping` every `xx` ms and disconnects you if nothing comes back within 2 seconds. Values of 200 or less turn it off. `0` turns it off. The timer applies to your connection only. Server mode only. The panel confirms with `_heartBeatTimer=xx`.

ASCII
```
HeartBeatTimer=3000
```

JSON
```
[{"Command":{"SetHeartBeatTimer":{"Value":3000}}}]
```

ping

Since 2017

Check the connection

The panel answers `ack`.

ASCII
```
ping
```

JSON
```
[{"FlowMessage":1}]
```

ack

Since 2017

Answer to a ping from the panel

nack

Accepted without action or reply. Use `ack` to answer the panel's heartbeat `ping`. `nack` (`FlowMessage` 3) is accepted too.

ASCII

```
ack
```

JSON

```
[{"FlowMessage":2}]
```

4.3 Panel settings

Clear

Since 2019

Clear all LEDs and displays

ClearLEDs ClearDisplays

`Clear` resets everything set with `HWC`, `HWCc`, `HWCx` and the displays. `ClearLEDs` only resets LEDs, `ClearDisplays` only displays.

ASCII

```
Clear
ClearLEDs
ClearDisplays
```

JSON

```
[{"Command":{"ClearAll":true}}]
[{"Command":{"ClearLEDs":true}}]
[{"Command":{"ClearDisplays":true}}]
```

PanelBrightness=x,y,z

Since 2020

Brightness of LEDs, displays and screen

Brightness 0-8 for LEDs (`x`), displays (`y`) and the backlight of an LCD touch screen (`z`, 0 = off). With one value it applies to LEDs and displays. Panels without a screen ignore the third value. Panels with a single brightness per component may use the average of `x` and `y`.

ASCII

```
PanelBrightness=8,6
```

JSON

```
[{"Command":{"PanelBrightness":{"OLEDs":6,"LEDs":8}}}]
```

DimmedGain=xx

Since ≤2020

Brightness of the dimmed state

Gain 1–64 for LEDs in the *dimmed* state, where 64 is full brightness. Values of 4–16 work well. `0` restores the panel default. The panel confirms with `DimmedGain=xx`.

ASCII

```
DimmedGain=12
```

JSON

```
[{"Command":{"SetDimmedGain":{"Value":12}}}]
```

SleepTimer=xx

Since 2019

Sleep after xx milliseconds without activity

SleepTimer?

Sets the time without operator activity before the panel goes to sleep; `0` disables sleep. `SleepTimer?` asks for the current value. The panel answers both with `_sleepTimer=xx`. GPIs and GPOs are not affected by sleep.

ASCII

```
SleepTimer=600000
SleepTimer?
```

JSON

```
[{"Command":{"SetSleepTimeout":{"Value":600000}}}]
[{"Command":{"GetSleepTimeout":true}}]
```

SleepMode=xx

Since 2019

How LEDs behave while the panel sleeps

`0` = animated LEDs (default), `1` = LEDs off.

ASCII

```
SleepMode=1
```

JSON

```
[{"Command":{"SetSleepMode":{"Mode":1}}}]
```

SleepScreenSaver=xx

Since 2019

What displays show while the panel sleeps

`0` = "Wake up on key press" message (default), `1` = "Sheep and goats" animation, `2` = "Save the OLEDs" message, `3` = keep content, just dim.

ASCII

```
SleepScreenSaver=3
```

JSON

```
[{"Command":{"SetSleepScreenSaver":{"Type":3}}}]
```

WakeUp!

Since 2019

Wake the panel from sleep

ASCII

```
WakeUp!
```

JSON

```
[{"Command":{"WakeUp":true}}]
```

Reboot

Since 2019

Restart the panel

The panel answers `Msg=Rebooting...` and restarts.

ASCII

```
Reboot
```

JSON

```
[{"Command":{"Reboot":true}}]
```

JSONNonOutbound=x

`JSONNonOutbound`

Since 2022

Have the panel reply in JSON

With `1`, everything the panel sends on this ASCII connection is a JSON array of `OutboundMessage` objects; `0` returns to plain text. Supported if the panel reports `JSONNonOutbound`.

ASCII

```
JSONNonOutbound=1
```

JSON

```
[{"Command":{"JSONconfig":{"Outbound":true}}}]
```

4.4 Status, network and registers

Connections?

Since ≤2020

List the connected clients

The panel answers with `_connections=` and the IP addresses of all clients.

ASCII

```
Connections?
```

JSON

```
[{"Command":{"GetConnections":true}}]
```

RunTimeStats?

Since ≤2020

Ask for uptime and boot count

The panel answers with `_bootsCount`, `_totalUptimeMin`, `_sessionUptimeMin` and `_screenSaverOnMin` (page 42).

ASCII

```
RunTimeStats?
```

JSON

```
[{"Command":{"GetRunTimeStats":true}}]
```

NetworkConfig?

NetworkSettings

Since 2024

Ask for the network settings

The panel answers with `_networkConfig={...}`. Supported if the panel reports `NetworkSettings`.

ASCII

```
NetworkConfig?
```

JSON

```
[{"Command":{"SendNetworkConfig":true}}]
```

SetNetworkConfig={json}

NetworkSettings

Since 2024

Change and apply the network settings

Fields: `dhcp`, `address`, `netmask`, `gateway`, `first_dns`, `second_dns`, `no_default_route`. The panel applies the settings immediately, so you will usually lose the connection.

ASCII

```
SetNetworkConfig={"dhcp":false,"address":"192.168.10.99","netmask":"255.255.255.0","gateway
    ↪ ":"192.168.10.1"}
```

JSON

```
[{"Command":{"SetNetworkConfig":{"address":"192.168.10.99","netmask":"255.255.255.0","gateway":"
    ↪ 192.168.10.1"}}}]
```

Registers?

Registers

Since 2022

List the panel's registers and their values

Registers are values the panel keeps beyond its hardware components, for example which tab of a touch interface is active. Their meaning depends on the panel. The panel answers with one line per register (page 44). Supported if the panel reports `Registers`.

ASCII

```
Registers?
```

JSON

```
[{"Command":{"SendRegisters":true}}]
```

MemX=n Flag#n=0/1 ShiftX=n StateX=n

Registers

Set a register

Since 2022

X is the register id (A--Z, 0--9). In JSON, Reg is 0 for Mem, 1 Flag, 2 Shift and 3 State. On UniSketch these commands set registers in a *remote* UniSketch controller running the TCP Server device core, not in the panel itself (Appendix A).

ASCII

```
ShiftA=1
```

JSON

```
[{"Registers":[{"Reg":2,"Id":"A","Value":1}]}]
```

4.5 TouchUI

These commands configure the widgets on touch-screen panels. Chapter 8 explains the concept. Supported if the panel reports TouchUI.

SetTouchUI={json}

TouchUI

Set or replace the widget configuration

Since 2026

The JSON is a TouchUIConfig: pages with widgets, each widget with a client-chosen HWC id. The panel adds the widgets to its topology and sends the new topology and map.

ASCII

```
SetTouchUI={"Pages":[{"Id":1,"Title":"Main","GridRows":2,"GridCols":3,"Widgets":[{"HWCID":201,"Type
    ↪ ":0,"Label":"Cut"},{"HWCID":202,"Type":2,"Label":"Gain","Col":1}]}]}
```

JSON

```
[{"Command":{"SetTouchUI":{"Pages":[{"Id":1,"Title":"Main","GridRows":2,"GridCols":3,
  "Widgets":[{"HWCID":201,"Label":"Cut"},{"HWCID":202,"Type":2,"Label":"Gain","Col":1}]}]}}}]
```

ClearTouchUI

TouchUI

Remove all widgets

Since 2026

The panel returns to its native topology and reports the widget HWCs as absent (map=id:0).

ASCII

```
ClearTouchUI
```

JSON

```
[{"Command":{"ClearTouchUI":true}}]
```

TouchUICapabilities?

TouchUI

Ask for screen size and supported widget types

Since 2026

TouchUIConfig?

`TouchUICapabilities?` is answered with `_touchUICapabilities={...}`. `TouchUIConfig?` is answered with the active configuration (`_touchUIConfig`) and page (`_touchUIActivePage`), which lets a reconnecting client pick up where it left off.

ASCII

```
TouchUICapabilities?
TouchUIConfig?
```

JSON

```
[{"Command":{"SendTouchUICapabilities":true}}]
[{"Command":{"SendTouchUIConfig":true}}]
```

SetTouchUIActivePage=n

TouchUI

Show another page

Since 2026

Switches to the page with id `n`; unknown ids are ignored. The panel confirms the change to all clients with `_touchUIActivePage=n`.

ASCII

```
SetTouchUIActivePage=2
```

JSON

```
[{"Command":{"SetTouchUIActivePage":{"PageId":2}}}]
```

HWCo#xx={json}

TouchUI

Overlay boxes on a widget, such as tracking frames on video

Since 2026

Replaces the set of boxes drawn on the widget. Coordinates are 0-1000 within the widget. An empty `Boxes` list clears them. Panels support at least 16 boxes per widget.

ASCII

```
HWCo#5={"Boxes":[{"Id":1,"X":100,"Y":200,"W":300,"H":250,"Label":"Face"}]}
```

JSON

```
{"HWCIDs":[5],"HWCOverlay":{"Boxes":[{"Id":1,"X":100,"Y":200,"W":300,"H":250,"Label":"Face"}]}}
```

HWCd#xx={json}

TouchUI

Change a widget's choices or range at runtime

Since 2026

Replaces the choice list of a dropdown (`Choices`, optional `Values` reported on selection) or the range of a slider or knob (`Min`, `Max`, `Step`) without resending the whole configuration, which would rebuild every widget.

ASCII

```
HWCd#5={"Choices":["Cam 1","Cam 2"],"Values":["1","2"]}
```

JSON

```
{"HWCIDs":[5],"HWCDomain":{"Choices":["Cam 1","Cam 2"],"Values":["1","2"]}}
```

4.6 Service and diagnostics

These commands are mainly used by SKAARHOJ for production, calibration and support.

HWCrawADCValues#xx=0/1

RawADCValues

Stream raw analog readings of a component

Since 2022

With `1` the panel continuously reports raw ADC values (`HWC#xx=Raw:n`) for the component. Supported if the panel reports `RawADCValues`.

ASCII

```
HWCrawADCValues#12=1
```

JSON

```
{"HWCIDs":[12],"PublishRawADCValues":{"Enabled":true}}
```

PublishSystemStat=xx

System

Report system statistics every xx seconds

Since 2021

The panel sends `SysStat=...` (page 43) at the interval; `0` turns it off. Supported if the panel reports `System`.

ASCII

```
PublishSystemStat=5
```

JSON

```
[{"Command":{"PublishSystemStat":{"PeriodSec":5}}}]
```

LoadCPU=xx

System

Stress test xx CPU cores

Since 2021

Loads 0-4 CPU cores for testing; `0` is off. Supported if the panel reports `System`.

ASCII

```
LoadCPU=2
```

JSON

```
[{"Command":{"LoadCPU":{"Level":2}}}]
```

SimulateEnvironmentalHealth=xx

EnvHealth

Test how your system reacts to a health warning

Since 2022

Makes the panel report `Normal`, `Safemode` or `Blocked` to all clients for 5 seconds (page 43). Supported if the panel reports `EnvHealth`.

ASCII

```
SimulateEnvironmentalHealth=
    ↪ Safemode
```

JSON

```
[{"Command":{"SimulateEnvironmentalHealth":{"RunMode":1}}}]
```

BurninProfile?

BurninProfile

Ask for the burn-in test profile

Since 2021

The burn-in profile drives an assisted test of every component, used in production and for remote checks. Answered with `_burninProfile={...}`. Supported if the panel reports `BurninProfile`.

ASCII

```
BurninProfile?
```

JSON

```
[{"Command":{"SendBurninProfile":true}}]
```

CalibrationProfile?

Calibration

Ask for the calibration profiles

Since 2022

SetCalibrationProfile={json}

`CalibrationProfile?` is answered with the current and the default profile. `SetCalibrationProfile` stores a new profile with the same structure as the default one. For SKAARHOJ calibration tools; supported if the panel reports `Calibration`.

ASCII

```
CalibrationProfile?
```

JSON

```
[{"Command":{"SendCalibrationProfile":true}}]
```

4.7 Legacy

ActivePanel=1

Legacy

Activate a UniSketch panel in client mode

Since 2017

Only needed in client mode: the panel sends `list` after connecting, and your server answers with `<NL>ActivePanel=1<NL>` (the leading newline guards against a partly received line after a reconnect). See Appendix A.3.

ASCII

```
ActivePanel=1
```

JSON

```
[{"Command":{"ActivatePanel":true}}]
```

Webserver=x

Legacy

Turn the UniSketch web server on or off

Since 2020

`1` = on, `0` = off. Only UniSketch panels have this web server.

ASCII

```
Webserver=1
```

JSON

```
[{"Command":{"SetWebserverEnabled":{"Enabled":true}}}]
```

encoderPressMode=xx

Legacy

How encoder presses are reported (UniSketch)

Since 2019

Bit 0 set (default): encoders report `Press` when pushed, and again after being held for one second. Deprecated: use the *Blue Pill Ready* device core option instead, which makes encoder presses behave like buttons (`Down`/`Up`, edge 16).

(unknowncommand)

Since 2019

Anything the panel does not understand

The panel answers `nack`. Some panels silently ignore malformed HWC commands.

5 Outbound Commands

From the panel to your system.

JSON examples show what a panel sends after `JSONonOutbound=1`: an array of `OutboundMessage` objects on one line. The binary encoding sends the same `OutboundMessage` structure.

5.1 Events

HWC#xx[.edge]=event

Since 2017

An operator used a hardware component

The most important message in the protocol. `xx` is the HWC number. The event part depends on the type of component:

Event	Sent by
`Down, Up`	Buttons, GPIs, touch widgets and pushed encoders: pressed and released.
`Abs:v`	Faders, T-bars, potentiometers, sliders: absolute position 0–1000.
`Speed:v`	Joysticks and other spring-loaded components: deflection −500 to 500, 0 at rest.
`Enc:v`	Encoders: number of steps since the last event; negative is counter-clockwise.
`Abs:x,y`	XY pads and video regions: several absolute values that belong together. JSON: `AbsoluteVector`.
`Speed:dx,dy`	XY pads in relative or joystick mode. JSON: `SpeedVector`.
`Text:string`	Text entered on the panel, e.g. an editable TouchUI label. The rest of the line is the value, which may be empty.
`Raw:v`	Raw ADC reading, after `HWCrawADCValues#xx=1`.
`Press`	Legacy (UniSketch): a press and release at once. Converted to `Down` plus `Up`.

Buttons with more than one active edge add `.edge`: 1 = top, 2 = left, 4 = bottom, 8 = right for two- and four-way buttons, and 16 when an encoder is pushed (*Blue Pill Ready* mode on UniSketch). A component without edges sends no suffix. The topology tells you what each component sends (`in`, page 66). Ranges can differ when the topology says so, for example `max=` or `steps=`.

ASCII
```
HWC#35.4=Down
```
JSON
```
[{"Events":[{"HWCID":35,"Binary":{"Pressed":true,"Edge":4}}]}]
```

ASCII
```
HWC#35.4=Up
```
JSON
```
[{"Events":[{"HWCID":35,"Binary":{"Edge":4}}]}]
```

ASCII
```
HWC#25=Abs:768
```
JSON
```
[{"Events":[{"HWCID":25,"Absolute":{"Value":768}}]}]
```

ASCII
```
HWC#42=Speed:136
```
JSON
```
[{"Events":[{"HWCID":42,"Speed":{"Value":136}}]}]
```

ASCII
```
HWC#38=Enc:-2
```
JSON
```
[{"Events":[{"HWCID":38,"Pulsed":{"Value":-2}}]}]
```

ASCII
```
HWC#50=Abs:300,700
```
JSON
```
[{"Events":[{"HWCID":50,"AbsoluteVector":{"Value":[300,700]}}]}]
```

ASCII
```
HWC#60=Text:Studio A
```

JSON
```
[{"Events":[{"HWCID":60,"Text":{"Value":"Studio A"}}]}]
```

ASCII
```
HWC#12=Raw:2048
```

JSON
```
[{"Events":[{"HWCID":12,"RawAnalog":{"Value":2048}}]}]
```

In binary and JSON, an event also carries a `Timestamp` in milliseconds and may carry `PrevValue`. A message can hold several events.

5.2 Availability map

`map=zz:xx`

Since 2017

Which hardware components are active

Sent for every HWC in reply to `map`, and by itself whenever availability changes. `zz` is the panel's own HWC number, the `id` in the topology. `xx` tells you:

xx	Meaning
`0`	Not available: an option that is not installed, a disabled component, or a panel that went to sleep.
`= zz`	Available, and events use the same number. This is the normal case.
`other`	Available, but events and feedback use HWC number `xx` (UniSketch configurations can remap components). Several components may share one number.
`bit 31`	Available but off-screen, for example a TouchUI widget on a page that is not shown. Mask it off to get the number.

Only show and drive components that are in the map. When a sleeping panel wakes up, its HWCs reappear in the map, which is your cue to send their feedback again.

ASCII
```
map=23:23
```

JSON
```
[{"HWCavailability":{"23":23}}]
```

ASCII
```
map=23:0
```

JSON
```
[{"HWCavailability":{"23":0}}]
```

5.3 Panel information

The reply to `list`. Every value is a separate line in ASCII and a field of `PanelInfo` in JSON and binary. Values that are empty are not sent.

`_model=string`

Since 2017

Model id

The model id has the form `NAMESPACE_MODEL_OPTION1_...`, separated by underscores. The namespace `SK` is for SKAARHOJ panels and `XP` for third-party hardware supported by SKAARHOJ; both are reserved. Use `CUSTOM` for your own panels. The model name uses `A--Z`, `0--9` and dashes. Options describe hardware variants that change the capabilities, for example `SK_RACKFUSIONLIVE_NKK` with NKK buttons. Older UniSketch firmware does not report options.

ASCII

```
_model=SK_RACKFUSIONLIVE
```

JSON

```
[{"PanelInfo":{"Model":"SK_RACKFUSIONLIVE"}}]
```

`_serial=string`

Since 2017

Serial number

`_version=string`

The serial number is alphanumeric. Use it, not the model, to identify a physical panel. `_version` is the software version.

ASCII

```
_serial=435775
_version=v2.4.1
```

JSON

```
[{"PanelInfo":{"Serial":"435775"}}]
[{"PanelInfo":{"SoftwareVersion":"v2.4.1"}}]
```

`_name=string`

Since 2021

Friendly name of the panel

The name the user gave the panel. Show it in your user interface.

ASCII

```
_name=Studio A Fusion
```

JSON

```
[{"PanelInfo":{"Name":"Studio A Fusion"}}]
```

_panelType=string

Since 2022

Kind of panel

Value	Meaning
`BPI`	Blue Pill Inside: SKAARHOJ hardware running Blue Pill. `PanelType` 1.
`Physical`	Other physical panels, such as Stream Deck or X-keys. 2.
`Emulation`	Software panels: Raw Panel Dummies, Raw Panel device core. 3.
`Touch`	Touch-screen panels. 4.
`Composite`	A panel composed of other panels. 5.

ASCII

```
_panelType=BPI
```

JSON

```
[{"PanelInfo":{"PanelType":1}}]
```

_platform=string

Since 2021

Software platform

Informational only. Known values: `ibeam` (Blue Pill), `simulator` (Raw Panel Dummies), `core` (device core), `xpanel` (external panels), `canvas`, `onejob` and `custom` (free to use for third-party panels).

ASCII

```
_platform=ibeam
```

JSON

```
[{"PanelInfo":{"Platform":"ibeam"}}]
```

`_support=flag,flag,...`

Since 2022

Features the panel supports

Check these flags before using optional features.

Flag	The panel supports
`ASCII, Binary`	The ASCII and binary encodings.
`JSONFeedback`	JSON states (`{...}` lines) on ASCII connections.
`JSONNonInbound`	JSON message arrays (`[...]` lines) on ASCII connections.
`JSONNonOutbound`	JSON replies after `JSONNonOutbound=1`.
`Processors`	Graphics processors (section 7.5). JSON and binary only.
`TouchUI`	TouchUI widget configuration (chapter 8).
`StateFeedback`	The `state` command.
`JogConfig`	The `HWCj` jog wheel command.
`Segments`	Individual LEDs with `HWCs` (added 2026-10).
`TextScroll`	Scrolling text in displays (section 7.3.2).
`Registers`	Registers (`Registers?`, `Mem`, `Flag`, ...).
`NetworkSettings`	`NetworkConfig?` and `SetNetworkConfig`.
`EnvHealth`	Environmental health reports.
`System`	`PublishSystemStat` and `LoadCPU`.
`RawADCValues`	`HWCrawADCValues`.
`BurninProfile`	`BurninProfile?`.
`Calibration`	Calibration profiles (only reported when calibration is unlocked).

ASCII

```
_support=ASCII,Binary,JSONFeedback,JSONNonInbound,JSONNonOutbound,Processors
```

JSON

```
[{"PanelInfo":{"RawPanelSupport":{"ASCII":true,"Binary":true,"ASCII_JSONfeedback":true,
  "ASCII_Inbound":true,"ASCII_Outbound":true,"Processors":true}}}]
```

`_bluePillReady=0/1`

Legacy

Since ≤2020

UniSketch: panel follows Blue Pill conventions

`1` if the *Blue Pill Ready* device core option is set: pushed encoders send `Down`/`Up` with edge 16, encoder LED rings behave like button LEDs, and the panel dims itself and sets a 2-minute sleep timer when no client is connected.

ASCII

```
_bluePillReady=1
```

JSON

```
[{"PanelInfo":{"BluePillReady":true}}]
```

`_serverModeLockToIP=ip;ip;...`

Since 2019

Server mode: IP addresses the panel accepts

`_serverModeMaxClients=n`

Sent when the panel is locked to specific clients, separated by semicolons. `_serverModeMaxClients` is the maximum number of simultaneous clients; `0` means no limit (UniSketch handles at most 7–8).

ASCII

```
_serverModeLockToIP
    ↪ =192.168.10.20
_serverModeMaxClients=4
```

JSON

```
[{"PanelInfo":{"LockedToIPs":["192.168.10.20"]}}]
[{"PanelInfo":{"MaxClients":4}}]
```

5.4 Topology

`_panelTopology_svgbase=svg`

Since 2019

Background drawing of the panel

`_panelTopology_HWC={json}`

The reply to `PanelTopology?`: an SVG drawing of the panel without components, and a JSON description of every component. Both are single lines in ASCII. Panels may also send them by themselves when the topology changes. See chapter 9.

ASCII

```
_panelTopology_svgbase=<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 3040 1858">...</svg>
_panelTopology_HWC={"HWc":[{"id":1,"x":201,"y":1449,"txt":"PRV 1","type":132},...],"typeIndex":{...}}
```

JSON

```
[{"PanelTopology":{"Svgbase":"<svg ...>...</svg>","Json":"{\"HWc\":[...],\"typeIndex\":{...}}"}}]
```

5.5 Flow control

ping

Since 2017

Heartbeat from the panel

Sent at the interval set with `HeartBeatTimer` (server mode) or about every 3 seconds (client mode). Answer with `ack` within 2 seconds.

ASCII

```
ping
```

JSON

```
[{"FlowMessage":1}]
```

ack

Since 2017

The panel received your ping

nack

`ack` answers your `ping`; UniSketch also sends `ack: image WxH received` after images. `nack` means the panel did not understand your last command (`FlowMessage` 3).

ASCII

```
ack
```

JSON

```
[{"FlowMessage":2}]
```

BSY

Legacy

Since 2017

Panel busy: pause sending

RDY

UniSketch flow control. After `BSY`, hold back new data until `RDY` or a `ping` arrives. Typically seen while sending images. Other implementations do not send these.

ASCII

```
BSY
RDY
```

JSON

```
[{"FlowMessage":4}]
[{"FlowMessage":5}]
```

list

Legacy

Client mode: the panel asks to be activated

Since 2017

In client mode the panel sends `list` after connecting and about every 60 seconds. Answer with `<NL>ActivePanel=1<NL>` and, optionally, the current feedback (Appendix A.3). In binary this is the `HELLO` flow message.

ASCII
```
list
```

JSON
```
[{"FlowMessage":100}]
```

5.6 Status and replies

_isSleeping=0/1

Since 2019

The panel went to sleep or woke up

Sent whenever the sleep state changes.

ASCII
```
_isSleeping=1
```

JSON
```
[{"SleepState":{"IsSleeping":true}}]
```

_sleepTimer=xx

Since 2019

Sleep timeout in milliseconds

Reply to `SleepTimer=` and `SleepTimer?`. `0` means sleep is disabled.

ASCII
```
_sleepTimer=600000
```

JSON
```
[{"SleepTimeout":{"Value":600000}}]
```

_heartBeatTimer=xx

Since 2019

Heartbeat interval in milliseconds

DimmedGain=xx

Confirmations of `HeartBeatTimer=` and `DimmedGain=`.

ASCII
```
_heartBeatTimer=3000
DimmedGain=12
```

JSON
```
[{"HeartBeatTimer":{"Value":3000}}]
[{"DimmedGain":{"Value":12}}]
```

`_connections=ip;ip;...`

Since ≤2020

Connected clients

Reply to `Connections?`.

ASCII

```
_connections=192.168.10.20;192.168.10.21
```

JSON

```
[{"Connections":{"Connection":["192.168.10.20","192.168.10.21"]}}]
```

`_bootsCount=n`

Since ≤2020

Runtime statistics

`_totalUptimeMin=n` `_sessionUptimeMin=n` `_screenSaverOnMin=n`

Reply to `RunTimeStats?`: number of boots, total uptime in minutes over the panel's life, uptime since the last boot, and the part of the total uptime spent asleep.

ASCII

```
_bootsCount=118
_totalUptimeMin=50211
_sessionUptimeMin=312
_screenSaverOnMin=20410
```

JSON

```
[{"RunTimeStats":{"BootsCount":118}}]
[{"RunTimeStats":{"TotalUptime":50211}}]
[{"RunTimeStats":{"SessionUptime":312}}]
[{"RunTimeStats":{"ScreenSaveOnTime":20410}}]
```

`Msg=string`

Since 2019

Information message

`ErrorMsg=string`

`Msg` is informational, for example `Msg=Rebooting...`. `ErrorMsg` reports a problem, for example a connection that is refused because of an IP lock or the client limit. Refusals are always sent in ASCII, since the panel does not know your encoding yet.

ASCII

```
Msg=Rebooting...
ErrorMsg=Max clients reached
```

JSON

```
[{"Message":{"Message":"Rebooting..."}}]
[{"ErrorMessage":{"Message":"Max clients reached"}}]
```

EnvironmentalHealth=xx

EnvHealth

Electrical noise around the panel

Since 2022

Panels with the `EnvHealth` flag watch their internal bus for errors caused by electrical or radio noise, and send this value when it changes and with the panel information.

Value	Meaning
`Normal`	No problems. `RunMode` 0.
`Safemode`	Some errors occurred. Nothing is compromised, but the panel checks every trigger more strictly. It should not last more than 10–15 seconds; if it does, or happens often, tell the user to move the panel, its power or network away from the noise source. `RunMode` 1.
`Blocked`	Too many errors to operate safely. The panel sends no events until the situation improves. Notify the user. `RunMode` 2.

The binary and JSON encodings also define `BusStatus` (`Fault`: true/false) for internal bus faults. It has no ASCII form; the Blue Pill Platform sends it when it detects a bus fault.

ASCII

```
EnvironmentalHealth=Safemode
```

JSON

```
[{"EnvironmentalHealth":{"RunMode":1}}]
```

SysStat=key:value:key:value:...

System

System statistics

Since 2021

Sent at the interval set with `PublishSystemStat`. Keys: `CPUUsage` (%), `CPUTemp` and `ExtTemp` (°C, −100 if not available), `CPUVoltage`, `CPUFreqCurrent/Min/Max`, `MemTotal`, `MemFree`, `MemAvailable`, `MemBuffers`, `MemCached` (kB), and the flags `UnderVoltage`, `FreqCap`, `Throttled`, `SoftTempLimit`, each also with a `Now` variant.

ASCII

```
SysStat=CPUUsage:12:CPUTemp:48.3:ExtTemp:31.0:...
```

JSON

```
[{"SysStat":{"CPUUsage":12,"CPUTemp":48.3,"ExtTemp":31}}]
```

MemX=n Flag#n=0/1 ShiftX=n StateX=n

Registers

Register values

Since 2022

Reply to `Registers?`, and sent when a register changes. Same format as the inbound command (page 28).

ASCII

```
ShiftA=1
```

JSON

```
[{"Registers":[{"Reg":2,"Id":"A","Value":1}]}]
```

HWCstate#xx[.edge]=value

StateFeedback

Current state of components

Since 2026

HWCMode#xx=yy HWCColor#xx=yy HWCExtended#xx=yy

Reply to `state`. `HWCstate` gives the physical state: `Down` or `Up` for buttons, `Abs:v` for faders, `Speed:v` for joysticks. Encoders have no state. `HWCMode`, `HWCColor` and `HWCExtended` report the feedback currently shown, with the same values as `HWC`, `HWCc` and `HWCx`; `HWCColor#xx=0` means the default color. Each value is its own line, so one component can take several lines, and there is no end-of-reply marker. In JSON and binary the input fields are optional: a missing field means not reported, so `Up`, `Abs:0` and `Speed:0` are real values.

Components and values the panel has nothing to report for are left out. A reply from a panel with one button held down and a fader at its minimum:

Reply to state

```
HWCstate#1=Down
HWCMode#1=36
HWCColor#1=132
HWCExtended#1=0
HWCstate#9=Abs:0
```

ASCII

```
HWCstate#25=Abs:768
HWCMode#38=36
```

JSON

```
[{"StateFeedback":[{"HWCID":25,"AbsoluteValue":768}]}]
[{"StateFeedback":[{"HWCID":38,"HWCMode":{"State":4,"Output":true
    ↪ }}]}]
```

5.7 TouchUI

_touchUICapabilities={json}

TouchUI

Screen size and supported widgets

Since 2026

_touchUIConfig={json}

Replies to `TouchUICapabilities?` and `TouchUIConfig?`. The capabilities contain the screen size in pixels, the preferred grid, limits and the supported widget types. The capabilities are sent again when a rotation changes the screen size.

ASCII

```
_touchUICapabilities={"ScreenWidth":1280,"ScreenHeight":720,"GridRows":4,"GridCols":6,...}
```

JSON

```
[{"TouchUICapabilities":{"ScreenWidth":1280,"ScreenHeight":720,"GridRows":4,"GridCols":6}}]
```

_touchUIActivePage=n

TouchUI

The page the panel shows now

Since 2026

Sent on every page change, whether the operator tapped a tab or a client sent `SetTouchUIActivePage`.

ASCII

```
_touchUIActivePage=2
```

JSON

```
[{"TouchUIActivePage":{"PageId":2}}]
```

5.8 Service

_networkConfig={json}

NetworkSettings

Network settings

Since 2024

Reply to `NetworkConfig?`, with the same fields as `SetNetworkConfig`.

ASCII

```
_networkConfig={"dhcp":true,"address":"192.168.10.99","netmask":"255.255.255.0","gateway
    ↪ ":"192.168.10.1"}
```

JSON

```
[{"NetworkConfig":{"dhcp":true,"address":"192.168.10.99","netmask":"255.255.255.0","gateway":"
    ↪ 192.168.10.1"}}]
```

`_burninProfile={json}`

`BurninProfile`

Burn-in test profile

Since 2021

`_calibrationProfile={json}` `_defaultCalibrationProfile={json}`

Replies to `BurninProfile?` and `CalibrationProfile?`, for SKAARHOJ test and calibration tools. In JSON they are `BurninProfile`, `CalibrationProfile` and `DefaultCalibrationProfile`, each with the profile as a string in `Json`. The current calibration profile overrides the default one entry by entry; an empty profile means the defaults apply.

ASCII

```
_burninProfile={...}
```

JSON

```
[{"BurninProfile":{"Json":"{...}"}}]
```

5.9 Legacy (UniSketch only)

`HWCmsg#xx=CN:a,b,c,d,e`

Legacy

Change notification

Since ≤2020

Sent by the UniSketch *Change Notification* action when one of its five parameters changes (Appendix A).

ASCII

```
HWCmsg#8=CN:0,8,0,0,64
```

`_state:reg=xx` `_shift:reg=xx`

Legacy

UniSketch register changes and setting echoes

Since 2019

`_sleepMode=xx` `_sleepScreenSaver=xx` `PanelBrightness=x,y` `Webserver=x`

UniSketch reports changes of its State and Shift registers, and confirms the settings above after they are set. Other implementations do not send these.

6 LEDs and Indicators

This chapter collects the reference tables for `HWC`, `HWCc` and `HWCx`.

6.1 Color palette

The palette used by `HWCc` (index plus 128), by `ColorIndex` in JSON, and by display text colors. The swatches show the nominal RGB value; actual LEDs differ.

#	Color	rrggbb	HWCc
0	Default	-	128
1	Off	000000	129
2	White	111111	130
3	Warm white	111101	131
4	Red*	110000	132
5	Rose	110101	133
6	Pink	110011	134
7	Purple	010011	135
8	Amber*	110100	136
9	Yellow*	111100	137

#	Color	rrggbb	HWCc
10	Dark blue	000011	138
11	Blue	000111	139
12	Ice	011011	140
13	Cyan	001111	141
14	Spring*	011100	142
15	Green*	001100	143
16	Mint	001101	144
17	Light gray†	101010	--
18	Dark gray†	010101	--

* Recommended for red/green bicolor buttons. † For color and grayscale displays only.

For colors outside the palette, use RGB mode: `HWCc#xx`$=192 + r \cdot 16 + g \cdot 4 + b$, with r, g, b from 0 to 3.

6.2 Blink patterns

Bits 8-11 of the `HWC` value select a blink pattern. Each bit enables a square wave at 4, 2, 1 or 0.5 Hz, and the LED is on whenever one of the enabled waves is on. Combinations give the patterns below.

Pattern	4 seconds, 125 ms per slot	`HWC` value (on)
1		292
2		548
4		1060
8		2084
3		804
6		1572
7		1828
5		1316
9		2340
10		2596
11		2852
13		3364
12		3108
14		3620
15		3876

6.3 VU meter thresholds

With `HWCx` type 4 (VU meter), each LED of a bar lights up when the value (0-1000) exceeds its threshold. The colors are fixed by the panel:

LEDs	Threshold per LED
6	>50 | >250 | >450 | >650 | >800 | >950
7	>50 | >200 | >350 | >500 | >650 | >800 | >950
10	>50 | >150 | >250 | >350 | >450 | >550 | >650 | >750 | >850 | >950
15	>50 | >116 | >182 | >250 | >316 | >382 | >450 | >516 | >582 | >650 | >710 | >770 | >830 | >890 | >950

Bars with 3 LEDs show one pattern across all three:

Value	LED 1, 2, 3
>50	
>250	
>450	
>650	
>800	
>950	

6.4 Legacy jog wheel configuration

Before `HWCj` existed, jog wheels were configured with `HWCx` type 7 (7×4096 + value). Use `HWCj` on panels that report `JogConfig`.

Value bits	Meaning
`10–11 = 0`	Bits 0–4: shuttle mode (0–31). Bit 5: automatic stop in shuttle. Bit 6: stop in shuttle.
`10–11 = 1`	Bits 0–7: value 0–255. Bits 8–9: which gap it sets: 0 = DETENTS_MIX2, 1 = FREE, 2 = FREE_W_ASSIST.

7 Displays

SKAARHOJ panels have many small displays, which we call *tiles*. Some are separate displays next to a button or encoder; others are areas of a larger shared display. You can fill a tile in three ways:

- **Text** with `HWCt`: the panel lays out title, value and labels to fit the tile. Easiest, and works on any tile size.
- **Bitmaps** with `HWCg`, `HWCgGray` and `HWCgRGB`: you control every pixel, so you must know the tile size.
- **Processors**: the panel renders images, meters, icons or international text from a short JSON description.

7.1 Tile sizes

The topology gives the exact pixel size of every tile (`disp`, page 66). Common sizes:

Size	Where
`64×32`	The standard tile, found on most panels. Content should always look reasonable at this size.
`112×32`	Wide tile, mostly above encoders.
`128×32`	Occasionally, e.g. the ID display of the RCP.
`64×48`	Taller tile with more room for text.
`52×24, 48×24`	Mini tiles used as button labels on large panels.
`256×20`	Wide title line on large panels.
`64×38, 86×48`	Color OLEDs in NKK SmartSwitch buttons (active area of 64×48 and 96×64 displays).
`96×64`	Color OLED.
`128×36, 128×72`	Tiles on Blue Pill displays.

When text tiles sit next to each other on a shared display, the panel renders text one pixel smaller in that direction and leaves a blank line between them. This *shrinking* does not apply to bitmaps.

7.2 Labels and values

The title bar of a tile tells the operator what kind of information they are looking at:

- A **solid bar** (`isLabel` = 0, the default) means the tile shows a *current value or state*, for example the source currently on an aux bus. Typical for encoders, toggle buttons and cycling buttons.
- A **thin line** under the title (`isLabel` = 1) means the tile shows a *label*: what will happen when the operator presses the button, for example the source that will be routed. Typical for ordinary buttons.



Value: solid bar



Label: thin line

7.3 Text format

`HWCt#xx=string` takes up to 21 fields separated by `|`. Most messages only use the first few; leave the rest out. The string is limited to 63 characters of printable ASCII. In JSON and binary the fields are named properties of `HWCText` (right column), which is easier to work with than counting pipes.

#	Field	Values	JSON / protobuf
0	Value	32-bit integer shown in the tile. Empty: no value (format 7). For formats 10 and 11: the font size.	`IntegerValue; TextStyling.Unformatted-FontSize for formats 10, 11`
1	Format	How values are printed: `0` integer, `1` float with 2 decimals from value/1000 (legacy), `2` percent, `3` dB, `4` frames, `5` 1/value, `6` Kelvin, `7` hidden, `8` float X.XXX (value/1000), `9` float XX.XX (value/100), `10` one big text line from the title (font size 1–4 in field 0), `11` two text lines from title and text 1 (font size 1–2), `12` float XXX.X (value/10).	`Formatting`
2	Icons	Bits 0–1: `1` fine (wave in the title bar), `2` lock, `3` no access. Bits 3–5 (×8): corner icon `1` cycle, `2` down, `3` up, `4` hold, `5` toggle, `6` OK, `7` question mark.	`StateIcon, ModifierIcon`
3	Title	Title text. Empty: no title bar.	`Title`
4	isLabel	`0` solid title bar (value), `1` thin line (label).	`SolidHeaderBar (inverted)`
5	Text 1	First text line, up to 24 characters. Alone, it is printed in a large font.	`Textline1`
6	Text 2	Second text line. When set, both lines use a small font. To get a small font for text 1 alone, set field 7.	`Textline2`
7	Value 2	Second value, printed after text 2 when used as a label/value pair.	`IntegerValue2`

#	Field	Values	JSON / protobuf
8	Pair mode	`1` label/value pairs (text 1 + value, text 2 + value 2), `2` upper pair boxed, `3` lower pair boxed, `4` both boxed.	`PairMode`
9	Scale	`1` strength bar, `2` center marker, `3` bar from center.	`Scale.ScaleType`
10, 11	Range	Low and high end of the scale.	`Scale.RangeLow, RangeHigh`
12, 13	Limits	Low and high limit markers. Set them to the range if you do not want them; they must be set.	`Scale.LimitLow, LimitHigh`
14	Scrolling	Text scrolling, see section 7.3.2. UniSketch and Link IO read this field as an image index.	`TextScroll`
15	Font face	Bits 0-2: text font, bits 3-5: title font (`0` default, `1` bold, `2` tiny). Bit 6: fixed width.	`TextStyling: TextFont.FontFace, TitleFont.FontFace, FixedWidth`
16	Font size	Bits 0-1: text width, 2-3: text height, 4-5: title width, 6-7: title height.	`TextStyling: TextFont/TitleFont .TextWidth, .TextHeight`
17	Spacing	Bits 0-1: title bar padding, bits 2-4: extra character spacing in pixels.	`TextStyling: TitleBarPadding, ExtraCharacterSpacing`
18	Inverted	`1`: render inverted.	`Inverted`
19	Pixel color	Color displays: palette index 1-18, or 64 + `rrggbb` for RGB.	`PixelColor`
20	Background	Color displays: as field 19.	`BackgroundColor`

Examples

Each tile below is a 64×32 display showing the result of the string beneath it. Use Raw Panel Explorer or `ColorDisplayButtonTest` (chapter 10) to try your own.



`32767`
Integer



`-9999`
Negative integer



`32767|1||Float2`
Float, 2 decimals



`299|2||Percent`
Percent



`999|3||dB`
dB



`1234|4||Frames`
Frames



`999|5||Reciproc`
Reciprocal



`9999|6||Kelvin`
Kelvin



`9999|7||[Empty!]`
Format 7: value hidden



`-3276|8||Float3`
Float, 3 decimals, fits ±9999



`||1|[Fine]|1`
Fine icon, title as label



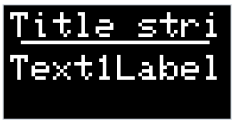
`||1|TitleString`
Title only, with fine icon



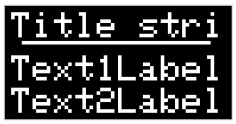
`|||TitleString|1`
Title as label



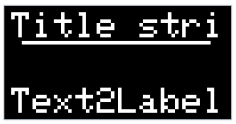
`|||Titlestring|1|Text1Label`
Text 1 in large font



`|||Titlestring|1|Text1Label||0`
Value 2 set: small font



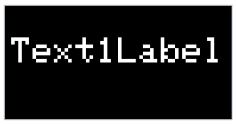
`|||Titlestring|1|Text1Label|Text2Label`
Two text lines



`|||Titlestring|1||Text2Label`
Text 2 only



`|||||Text1Label`
No title, large text



`|||||Text1Label||0`

No title, small text



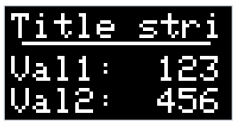
`|||||Text1Label|Text2Label`

No title, two lines



`|||||Text2Label`

No title, text 2 only



`123|||Titlestring|1|Val1:|Val2:|456`

Label/value pairs



`-1234|1||Coords:||x:|y:|4567|2`

Upper pair boxed



`-1234|1||Coords:||x:|y:|4567|3`

Lower pair boxed



`-1234|1||Coords:||x:|y:|4567|4`

Both pairs boxed



`-500|1||Coords:||||||1|-1000|1000|-700|700|1`

Strength scale



`-500|1||Coords:||||||2|-1000|1000|-700|700|2`

Center marker scale

Text scrolling

Field 14 makes text that is too long for the tile scroll, by whole characters so the formatting is kept. Only send it to panels that report `TextScroll`. UniSketch and Link IO read field 14 as the old image index and may show a stray icon; panels that know neither ignore it. In JSON it is the `TextScroll` object of `HWCText`.

Bits	Meaning
`0–1`	Mode (`ScrollMode`): `0` off, `1` loop, `2` bounce.
`2, 3, 4`	Scroll the title, text 1, text 2 (`ScrollTitle`, `ScrollTextline1`, `ScrollTextline2`).
`5–6`	Speed (`ScrollSpeed`): `0` default, `1` slow, `2` normal, `3` fast.
`7–8`	Pause at the ends (`PauseDwell`): `0` default, `1` short, `2` medium, `3` long.
`9`	Adaptive speed (`AdaptiveSpeed`): all fields start and finish together.

ASCII

```
HWCt#3=|||Title|1|T1|T2|||||||541
```

JSON

```
{"HWCIDs":[3],"HWCText":{"Formatting":7,"Title":"Title","Textline1":"T1","Textline2":"T2","PairMode"
    ↪ :1,
  "TextScroll":{"ScrollMode":1,"ScrollTitle":true,"ScrollTextline1":true,"ScrollTextline2":true,"
    ↪ AdaptiveSpeed":true}}}
```

Value 541 = 512 + 16 + 8 + 4 + 1: loop mode, scrolling title and both text lines, with adaptive speed.

7.4 Bitmaps

Always send bitmaps in the exact pixel size of the tile (from the topology). This is the reference behavior and the only one that works the same on all panels. Smaller or larger images are handled differently by different implementations: a smaller image may be centered or placed in the top left corner, and a larger one may be cropped or drawn incorrectly.

Command	Pixel format
`HWCg`	Monochrome, 1 bit per pixel. Each row starts on a new byte; the most significant bit is the leftmost pixel; 1 = on. A 64×32 image is 256 bytes.
`HWCgGray`	Grayscale, 4 bits per pixel, two pixels per byte (high nibble first), rows packed without padding.
`HWCgRGB`	Color, 16 bits per pixel, rows packed without padding. Each pixel is a 16-bit word sent most significant byte first: bits 15–11 blue, 10–5 green, 4–0 red, that is $(B \ll 11) \mid (G \ll 5) \mid R$. This is *not* the usual RGB565 with red in the top bits; decoding it as RGB565 swaps red and blue. The same bytes are used in `HWCGfx.ImageData` in JSON and binary.

JSON and binary

One state carries the whole image in `HWCGfx`. `W` and `H` must always be set and match the data length; otherwise the image is ignored. `XYoffset`, `X` and `Y` place an image at a pixel position within the tile, but several implementations ignore them. Do not rely on the offset.

JSON

```
{"HWCIDs":[34],"HWCGfx":{"ImageType":2,"W":2,"H":2,"ImageData":"8A8="}}
```

ASCII

ASCII lines should stay below about 250 characters, so the base64 data is split over several lines (about 170 bytes of image data each). The first line, sequence 0, carries the header; the rest only their sequence number. The panel renders the image when the last line arrives. Always send the lines in order and without other commands for the same HWC in between.

Format

```
HWCg#xx=0/last,WxH[,x,y]:base64
HWCg#xx=1:base64
...
HWCg#xx=last:base64
```

Example: 64x38 monochrome, 16 lines

```
HWCg#7=0/15,64x38:////////////////////8QhCA==
HWCg#7=1:QhCEIQvEIQhCEIQhC/////////w==
HWCg#7=2:///EIQhCEIQhC8QhCEIQhCELxA==
...
HWCg#7=15:hCEL/////////////////////w==
```

The function `InboundMessagesToRawPanelASCIIstrings()` in rawpanel-lib does this splitting for you. A special legacy form for 64×32 images on UniSketch uses three lines with only the index as header (`HWCg#xx=0:...`, `1:`, `2:` with 86, 86 and 84 bytes).

> **Frame rates.** Color images carry 16 times more data than monochrome ones. Blue Pill panels can show moving images on a few tiles at a fair frame rate. UniSketch panels manage about six 96×64 color images per second.

Raw Panel Explorer converts an uploaded image to the exact size of any tile and shows the resulting commands. The command line tool `ImageConverter` in raw-panel-utils does the same with more control over scaling.

7.5 Processors

Processors let the panel do the rendering. You send a short description in the `Processors` field of a state, and the panel generates the bitmap. They are available on panels that report `Processors`, and only in JSON and binary. `W` and `H` must match the tile size from the topology.

GfxConv: send an ordinary image

Send a PNG, JPEG, GIF or WebP image as base64 in `ImageData`; the type is detected automatically. Optional fields: `ImageType` (`0` mono, `1` RGB, `2` gray), `W` and `H`, `Scaling` (`0` none, `1` fill, `2` fit, `3` stretch), and `Filters` (the image filter syntax also used in Reactor).

JSON

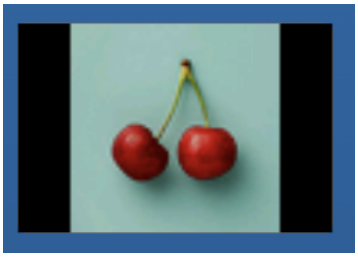
```
{"HWCIDs":[1],"Processors":{"GfxConv":{"W":96,"H":64,"Scaling":2,"ImageType":1,"ImageData":"/9j/4
    ↪ QAYRXhpZgAASUkqAAgAAAAA..."}}}
```



Source: JPEG, 200×200



Default: mono, stretched



RGB, 96×64, fit (example above)

AudioMeter: level meters

Draws a mono or stereo audio meter from values 0–1000. Fields: `Data1`/`Data2` (levels), `Peak1`/`Peak2`, `Mono`, `Title`, `RangeMapping` and `RMYAxis` (below), and `MeterType`: `0` free size, `1` and `2` fixed 176×32 designs with a scale for the meter displays of Wave Board panels.

JSON

```
{"HWCIDs":[37],"Processors":{"AudioMeter":{"Title":"My Title String","W":176,"H":32,"Data1":500,"Peak1
    ↪ ":700,"Mono":true}}}
```



Mono meter with title and peak, 176×32

JSON

```
{"HWCIDs":[37],"Processors":{"AudioMeter":{"MeterType":1,"Title":"My Title String","W":176,"H":32,"
    ↪ Data1":500,"Peak1":700,"Data2":400,"Peak2":1000}}}
```



Stereo, without `Mono`, free size



Stereo on a 64×48 tile



Type 1



Type 2

StrengthMeter: a bar with title and value

Draws a horizontal bar for `Data1` (0-1000) with a `Title` and a `ValueString`, for example the position of a T-bar.

JSON

```
{"HWCIDs":[21],"Processors":{"StrengthMeter":{"W":256,"H":32,"Title":"T-Bar position","ValueString":"
    ↪ 20.3%","Data1":810}}}
```



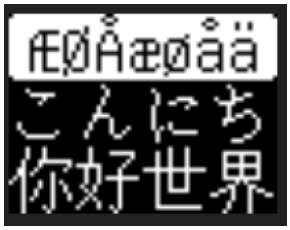
Strength meter, 256×32

UniText: international text

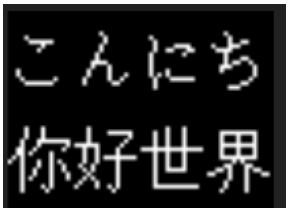
Renders a title and up to four text lines in UTF-8 with a Unicode font, for languages the `HWCt` format cannot show. Each line needs 16 pixels of height, so a 32-pixel tile fits two lines. Fields: `Title`, `SolidHeaderBar`, `Textline1`–`Textline4`, and `Align` (`0` center, `1` left, `2` right). Without a title only the text lines are drawn. The example uses JSON escapes for the non-ASCII characters; plain UTF-8 works as well.

JSON

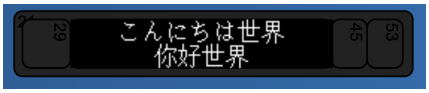
```
{"HWCIDs":[2],"Processors":{"UniText":{"W":64,"H":48,"Title":"\u00c6\u00d8\u00c5\u00e6\u00f8\u00e5\
    ↪ u00e4",
  "SolidHeaderBar":true,"Textline1":"\u3053\u3093\u306b\u3061\u306f\u4e16\u754c","Textline2":"\u4f60\
    ↪ u597d\u4e16\u754c"}}}
```



64×48, solid title bar



Two lines, no title



Title and one line

Icon: Material Icons

Renders one glyph from Google's Material Icons font. `GlyphCode` is the code point (hexadecimal or decimal) from fonts.google.com/icons. `IconType` is `0` mono, `1` RGB or `2` gray. `Background` draws a rounded box with `BackgroundRadius`. `GlyphSize` is calculated automatically when left out.



JSON

```
{"HWCIDs":[2],"Processors":{"Icon":{"W":64,"H":48,
  "GlyphCode":"e92b","IconType":1,
  "Background":true,"BackgroundRadius":12}}}
```

TextToGraphics and Test

`TextToGraphics` renders the `HWCText` of the same state into a bitmap of size `W×H`; it is rarely needed. `Test` draws a test pattern showing the given size.





JSON

```
{"HWCIDs":[2],"HWCText":{"Title":"Hey!","SolidHeaderBar":true,"
    ↪ Textline1":"Count: 1470"},
  "Processors":{"TextToGraphics":{"W":64,"H":48}}}
```

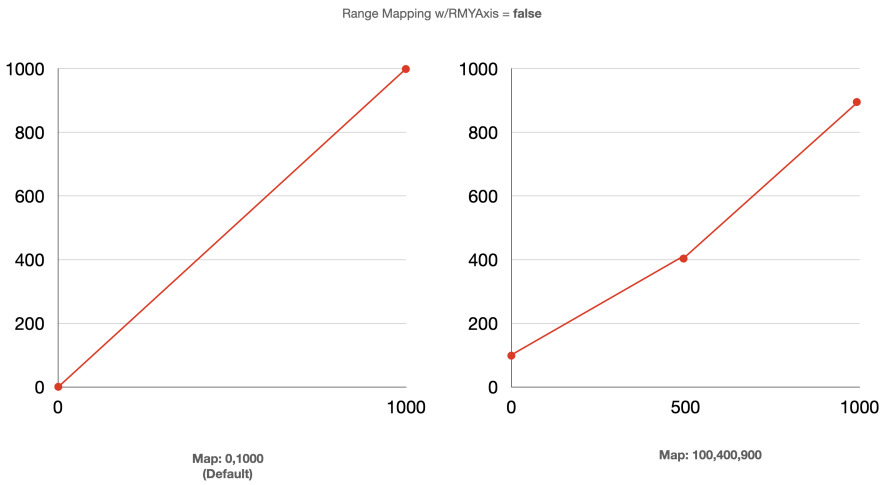
JSON

```
{"HWCIDs":[2],"Processors":{"Test":{"W":64,"H":48}}}
```

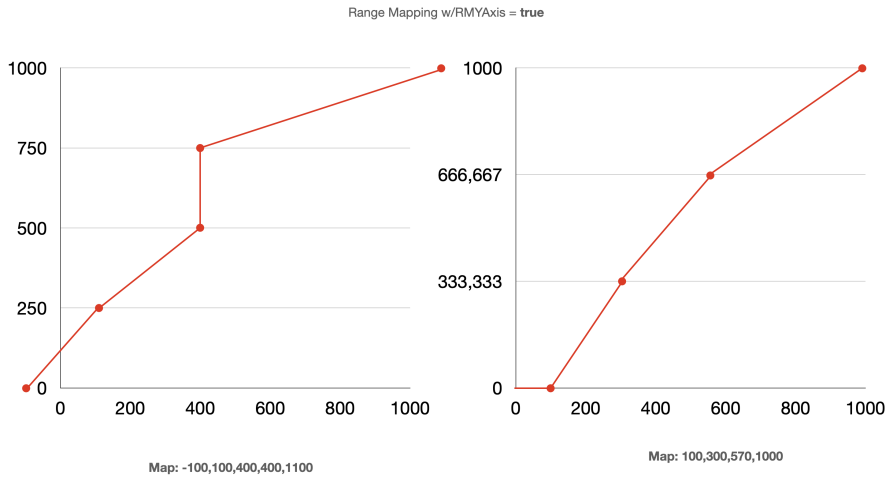
Range mapping

AudioMeter and StrengthMeter draw their bars linearly. `RangeMapping` bends that line, for example to match a dB scale in a background image. It is a comma-separated list of output values for evenly spaced input values. With `"0,400,1000"` the input 0–500 maps to 0–400 and 500–1000 to 400–1000. With `"0,300,700,900"` the input is split in thirds: 0–333 maps to 0–300, 333–667 to 300–700 and 667–1000 to 700–900.

With `RMYAxis` set to `true` the list is read the other way round: it gives the input values for evenly spaced output values. `"0,300,700,900"` then maps input 0–300 to output 0–333, 300–700 to 333–667 and 700–900 to 667–1000.



Range mapping with `RMYAxis` false: the default `0,1000` (left) and `100,400,800` (right)



Range mapping with `RMYAxis` true

8 TouchUI

Touch-screen panels can show widgets that your system defines: buttons, sliders, meters, video and more. Each widget gets an HWC number that you choose, and from then on it behaves like any other hardware component. It sends the usual events and accepts the usual feedback, so the rest of your integration does not change. Supported on panels that report `TouchUI`.

8.1 How it works

1. Ask for `TouchUICapabilities?` to learn the screen size, preferred grid and supported widget types.
2. Send `SetTouchUI={json}` with pages and widgets. Pick HWC numbers that do not collide with the panel's own HWCs.
3. The panel adds the widgets to its topology and sends the new `_panelTopology_HWC` and the map.
4. Drive the widgets with `HWC`, `HWCc`, `HWCt`, `HWCx` and `HWCg` as usual, and handle their events.

Example

```
→ SetTouchUI={"Pages":[{"Id":1,"GridRows":2,"GridCols":3,"Widgets":[
→   {"HWCID":201,"Type":0,"Label":"Cut"},
→   {"HWCID":202,"Type":2,"Label":"Gain","Col":1}]}]}
← _panelTopology_HWC={...}                          now includes HWC 201 and 202
← map=201:201
← map=202:202
→ HWCc#201=132
→ HWC#201=36                                        the Cut button lights up red
→ HWCx#202=20980                                    the Gain slider moves to 500
← HWC#201=Down                                      the operator taps Cut
← HWC#202=Abs:640                                   ...and drags the slider
```

The `SetTouchUI` line is shown wrapped; send it as one line.

8.2 Pages and layout

A configuration has one or more **pages**; with more than one, the panel shows tabs. Each page lays out its widgets either in a **grid** (`GridRows`, `GridCols`; widgets use `Row`, `Col`, `RowSpan`, `ColSpan`) or in **pixels** (`X`, `Y`, `W`, `H`). Grid layouts adapt to any screen and to rotation, so prefer them.

Widgets on a page that is not shown are reported in the map with bit 31 set (`map=201:2147483849`). The page itself is reported with `_touchUIActivePage`, which is the reliable way to track it. Switch pages with `SetTouchUIActivePage`.

8.3 Widget types

Type	Sends / accepts
0 BUTTON	Momentary button: Down/Up. FourWay adds edges. Accepts HWC, HWCc, HWCt, HWCg.
1 TOGGLE	Latching button: Down/Up. Accepts HWC, HWCc, HWCt.
2 SLIDER	Linear fader: Abs:0--1000. Accepts HWCx type 5 (position) and HWCt.
3 KNOB	Rotary dial: Abs:0--1000. Like the slider.
4 METER	Passive meter. Accepts HWCx types 1 and 4, HWCg.
5 LABEL	Text tile; sends Down/Up on tap. With EditKind the operator can type text, sent as Text:.... Accepts HWCt, HWCc.
6 IMAGE	Image tile; sends Down/Up on tap. Accepts HWCg.
7 VIDEO	Live video from Source; sends the touched position as Abs:x,y. Overlay boxes with HWCo.
8 ENCODER	Encoder pad: Enc:±1 for −/+, and Down/Up with edge 16 for press.
9 DROPDOWN	Choice list: Abs:index, and Text:value when HWCd gives values.
10 XYPAD	2D surface: Abs:x,y, or Speed:dx,dy when Relative or Joystick is set.
11 COMPRESSOR	Draws a dynamics curve from the parameters in Options.Params, each with its own HWC.
12 EQUALIZER	Draws an EQ curve from the bands in Options.EqBands, each parameter with its own HWC.

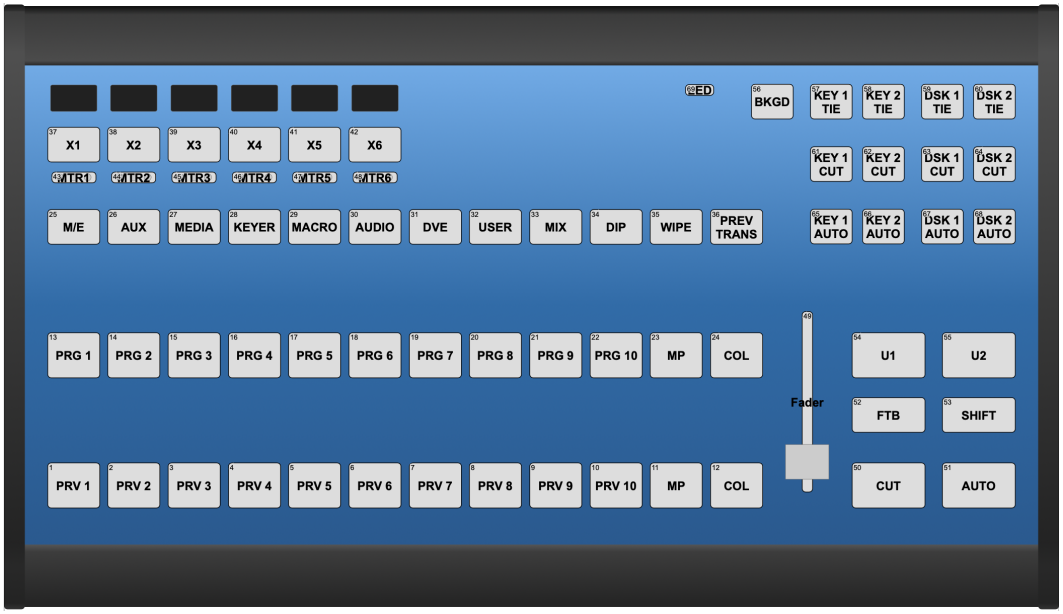
Widget options (Options) cover ranges, orientation, accent color, visual variants, video scaling and more. Panel-wide options (TouchUIConfig.Options) control debug overlays, the position of the page selector and configuration menu, and the display orientation. The complete list with comments is in ibeam-rawpanel.proto, messages TouchUIConfig, TouchUIWidget, TouchUIWidgetOptions and TouchUIGlobalOptions.

8.4 Several clients

Every client receives the topology, map and active page when a configuration changes. A client that connects later sends TouchUIConfig? to get the active configuration and page. ClearTouchUI removes all widgets and returns to the panel's native topology.

9 Panel Topology

`PanelTopology?` returns everything you need to draw a panel and to understand its components: an SVG background and a JSON list of hardware components with position, size, input and output types and display size. Your system can support any panel, including future ones, by reading the topology instead of hard-coding models.



An Air Fly panel drawn from its topology: the SVG background plus one shape per component.

9.1 SVG background

`_panelTopology_svgbase` is a complete SVG document with the panel's chassis and section markings, in units of 0.1 mm. Draw the components on top of it. The rendering reference is the `topology` package in rawpanel-lib, and Raw Panel Explorer shows the result as an interactive panel view.

9.2 JSON components

`_panelTopology_HWC` has two main keys: `HWc`, a list of components, and `typeIndex`, reusable component types that the list refers to. (A `title` key may exist; ignore it.)

Topology (excerpt)


```
{"HWc":[
   {"id":1,"x":201,"y":1449,"txt":"PRV 1","type":132},
   {"id":9,"x":180,"y":909,"txt":"Knob A","type":15,
    "typeOverride":{"disp":{"w":64,"h":32,"subidx":0},
      "sub":[{"_":"r","_x":-55,"_y":-550,"_w":190,"_h":95,"rx":5,"ry":5}]}}],
 "typeIndex":{
   "15":{"w":160,"out":"rgb","in":"pb","desc":"Encoder"},
   "28":{"w":30,"h":710,"in":"av","ext":"pos","subidx":0,"desc":"Motorized Fader 60mm",
         "sub":[{"_":"r","_x":-63,"_y":53,"_w":125,"_h":250}]},
   "36":{"w":570,"h":151,"disp":{"w":128,"h":32},"desc":"OLED Display Tile"}}}
```


Type 15 has only `w`, so it is drawn as a circle 16 mm across. It accepts RGB color and sends encoder pulses and presses. Type 28 has `w` and `h`, so it is a rectangle; it is a vertical absolute component (`av`) that can position itself (`ext:"pos"`), and its `sub` element is the fader knob. Type 36 is a display with a 128×32 tile.

Component keys (`HWc`)

Key	Meaning
`id`	HWC number. This is the `zz` in `map=zz:xx`. Only show components that are in the map.
`x, y`	Center of the component on the SVG, in 0.1 mm from the top left.
`txt`	Default label. `|` is a line break.
`type`	Key into `typeIndex`. Only valid within this topology: the same number can mean a different type in another panel's topology, or after an update.
`typeOverride`	Type keys that override the type for this component only. Numbers (`w`, `h`, `subidx`) override when larger than zero, strings (`in`, `out`, `ext`, `desc`, `render`) when not empty, `rotate` when not zero, `disp` and `sub` when they contain values.
`UIparent`	Id of a component this one moves with in a simulation.
`UIyang`	For `iv`/`ih` components: the id of the other axis of the same joystick.

Type keys (typeIndex)

Key	Meaning
w, h	Size in 0.1 mm. Only w: a circle with diameter w. Both: a rectangle.
in	What the component sends: b button, b4 four-way button, b2v/b2h two-way button (vertical/horizontal), gpi GPI, pb encoder with push, p encoder without push, pi encoder with intensity (jog), av/ah/ar/a absolute (vertical, horizontal, rotary, unspecified), iv/ih/ir/i intensity (joysticks). Parameters may follow after commas, see below.
out	What it shows: gpo output, mono single-color LED, rg red/green, rb red/blue, rgb full color.
ext	Extended output: pos self-positioning (motorized fader), steps LED bar (one sub element per LED, ordered by _idx), jogcfg jog wheel configuration.
disp	Display: w and h in pixels; type gray or color (default monochrome, white on black), or text for text-only displays; subidx points to the sub element that marks the display.
sub	Extra shapes to draw: "_":"r" rectangle (_x, _y top left relative to the center, _w, _h), "c" circle (_x, _y center, r), "d" display area for simulators. Other keys become SVG attributes.
subidx	The sub element with a special role: the fader knob, the lit part of a GPO, or the display. 0 if missing; −1 for none.
render	Always draw: txt the label, invtxt in light color, hwcid the id.
desc	Description.

Parameters of in, for example "ar,steps=16":

Parameter	Meaning
steps=n	The component has n positions spread over its range. Two steps give 0 and 1000 (absolute) or −500, 0 and 500 (intensity).
stepList=a;b;...	The exact step values. Takes precedence over steps.
max=n	Largest value. Default 1000 for absolute and 500 for intensity components.
dim=n	The component also sends a vector of n values, for example a joystick or track ball.

9.3 Topologies change

Treat a topology as data that belongs to one panel at one moment:

- Ask for the topology when you connect, and use any topology the panel sends later instead. TouchUI configurations, for example, change it.
- Panels with the same model id may have different topologies. Cache a topology per serial number, not per model.
- Follow the map: show and hide components as their availability changes. A panel that goes to sleep removes its components from the map; you may prefer not to redraw in that case.
- Draw components only from the data in the topology. Do not assume that a type number means the same thing in another topology.

10 Tools and Libraries

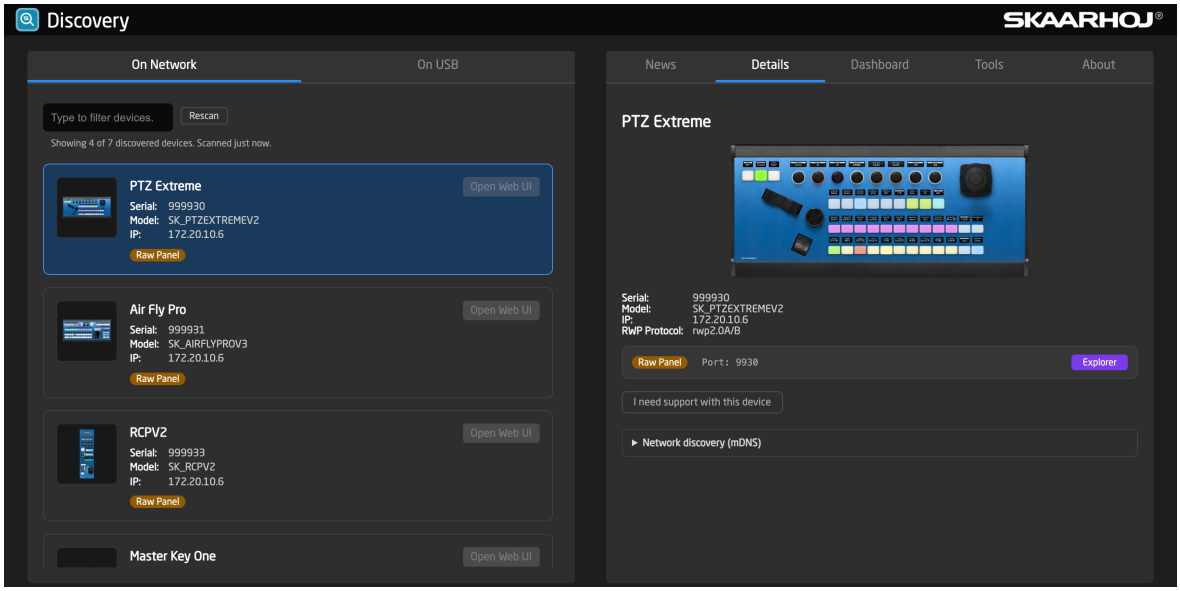
With the tools below you can learn and test the Raw Panel Protocol without any hardware: Raw Panel Dummies emulates a panel, and Raw Panel Explorer or a terminal connects to it.

10.1 Raw Panel Explorer

Raw Panel Explorer is a tool in **SKAARHOJ Discovery**, the SKAARHOJ desktop app for Windows, macOS and Linux that finds SKAARHOJ devices on the network and on USB. Download it from skaarhoj.com.

Raw Panel Explorer is an interactive Raw Panel client. It connects to a panel, draws the panel from its topology, shows every event, and lets you set LEDs, colors, text and images on any component. It shows the exact ASCII and JSON form of everything it sends, which makes it the quickest way to learn the protocol.

To open it, turn on **Expert** mode in the footer of Discovery. Every panel under **On Network** that announces a Raw Panel server then has an **Explorer** button next to that service in its **Details** tab. To connect to a server that is not listed, for example Raw Panel Dummies on another computer, enter its IP address and port under **Tools**, **Raw Panel Explorer**.



SKAARHOJ Discovery in Expert mode, listing four panels emulated by Raw Panel Dummies (section 10.2). The Details tab of the selected PTZ Extreme has an Explorer button next to its Raw Panel service.

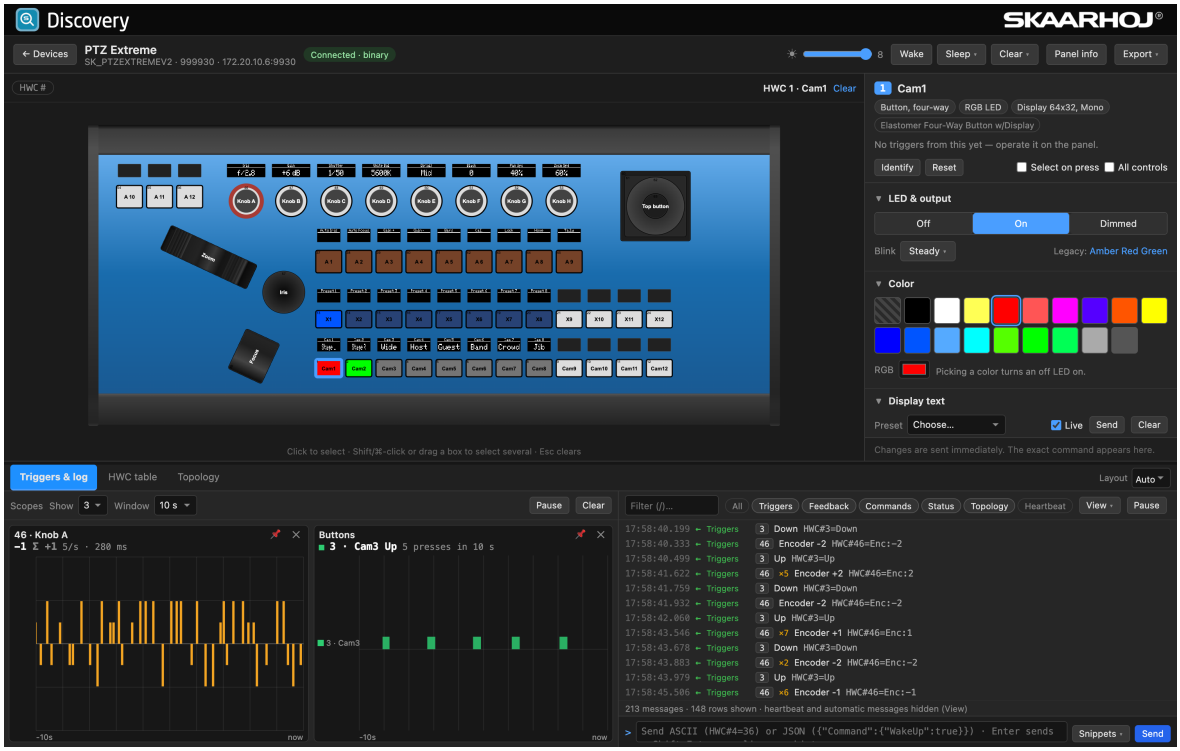
The Explorer window has three parts:

- **Panel**: the panel drawn from its topology, with the current LED colors, text and images. Click a component to select it, or Shift-click or drag a box to select several.
- **Inspector**: feedback controls for the selected components: LED state and color, blink mode, display

text and images. Changes are sent at once.

- **Dock**: *Triggers & log* has scopes that plot faders, joysticks, encoders and buttons as you operate them, and a log of every message in and out. *HWC table* lists all components with their types and properties, and *Topology* shows the raw topology. A command line at the bottom sends any ASCII or JSON command.

The toolbar sets the brightness, wakes the panel or sends it to sleep, clears it, shows the panel info, and exports the HWC table to Excel or CSV.



Raw Panel Explorer connected to a PTZ Extreme. The camera buttons, presets and knob displays have been given colors and text, and button Cam1 is selected in the inspector. The scopes plot Knob A turning and Cam3 being pressed, and the log lists the same triggers.

10.2 Raw Panel Dummies

The opposite of Raw Panel Explorer: it emulates any SKAARHOJ panel in your web browser and offers a real Raw Panel server for it. Use it to develop before the hardware arrives, or to test panels you do not have. Downloads: github.com/SKAARHOJ/raw-panel-dummies-releases. With the option `-logIncomingMessagesAsJson` it prints every message it receives as JSON, which shows you the JSON form of any ASCII command.

10.3 rawpanel-lib

The reference implementation in Go, at github.com/SKAARHOJ/rawpanel-lib:

Part	Content
`ibeam-rawpanel-proto/`	`ibeam-rawpanel.proto`, the definition of all messages. Generate code for other languages from it with `protoc`. Generated C++ is included.
`converterFunctions.go`	Conversion between ASCII and protobuf: `RawPanelASCIIstringsToInboundMessages()`, `InboundMessagesToRawPanelASCIIstrings()` and the outbound equivalents.
`gorwp/`	A high-level Go API: connect, bind handlers to events, set colors, text and images.
`topology/`	Parsing and rendering of topologies.
`tools/rwp-converter`	Command line tool that converts between ASCII and JSON, in both directions.

rwp-converter

```
$ echo 'HWCc#38=137' | rwp-converter -mode inbound -compact
{"States":[{"HWCIDs":[38],"HWCColor":{"ColorIndex":{"Index":9}}}]}
```

10.4 raw-panel-utils

Command line utilities with Go source code, useful both as tools and as examples, at github.com/SKAARHOJ/raw-panel-utils.

Utility	Purpose
`ColorDisplayButtonTest`	Cycles colors and display content on a panel and prints all events. With `-verboseIncoming 2` it also prints every message it sends as JSON. A good all-round test tool.
`ServerPanel2ClientSystem`	Connects to a panel in binary or ASCII and offers it again as a TCP server on another port, in binary or ASCII. Turns a binary panel into an ASCII one, or gives a UniSketch panel JSON and binary.
`ServerPanel2ServerSystem`	Connects to a panel and to a system that is itself a TCP server, and bridges the two, converting the encoding if needed.
`ImageConverter`	Converts PNG and JPEG files into Raw Panel image commands.
`Burnin`	Runs a panel's burn-in test sequence.

ServerPanel2ClientSystem

```
$ ServerPanel2ClientSystem -binPanel 192.168.11.8:9923 9924
$ nc localhost 9924
list
_model=SK_RACKFUSIONLIVE
_serial=435775
...
```

The first command connects to a binary panel and offers an ASCII server on port 9924. The second connects to it with `nc`.

10.5 Raw Panel Bridge

Connects panels to a remote system that cannot reach them, for example a cloud service. It runs on a computer on the panel's network or directly on a skaarOS device, connects to the panel locally and opens an outbound WebSocket or TCP connection to the remote system.

11 Implementation Support

Which implementation supports which feature. ● supported, ◐ partly (see the numbered note), ○ not supported, – not needed: the hardware has no use for it, or it is a legacy or SKAARHOJ service feature, ? not determined. Link IO covers several products; a feature counts as supported when it works on the products that have the hardware for it. Features that have a `_support` flag are only available when the panel announces the flag.

The Blue Pill Platform column is the Hardware Manager, the component that runs Raw Panel on Blue Pill panels. Stream Deck, X-keys, Touch, HIDs and Raw Panel Dummies are built on the `rawpanel-server` Go library, which handles the connection, encodings, client limits and heartbeat. Touch is the touch-screen panel driver. The HIDs column covers the HID device driver (keyboards, mice, game pads, jog/shuttle controllers and similar) and the Blackmagic focus and zoom demands. They mainly send input events; display and LED states are ignored, except for the one LED on the zoom demand. Other external panel drivers use the same library, but each has its own protocol handling and is not covered here. The matrix is a snapshot made from the source code of each implementation, listed at the end of the chapter. It is refreshed from time to time; the implementations themselves are the final word.

Feature	Blue Pill Platform	Link IO	Stream Deck	X-keys	Touch	HIDs	Raw Panel Dummies	UniSketch (legacy)
TRANSPORT AND ENCODINGS								
Plain-text ASCII	●	●	●	●	●	●	●	●
Binary (protobuf)	●	○	●	●	●	●	●	○
ASCII/binary auto-detection	●	–	●	●	●	●	●	–
JSON states ({...} lines)	●	○	●	●	●	●	●	○
JSON messages ([...] lines)	◐[1]	○	●	●	●	●	●	○
JSON replies (JSONonOutbound=1)	●	○	●	●	●	●	●	○
Server mode	●	●	●	●	●	●	●	●
Client mode	○	◐[2]	○	○	○	○	○	●
Several clients	●	●	●	●	●	●	●	●
Max clients / Lock to IP	●	●	●	●	●	●	●	●
HeartBeatTimer	●	●	●	●	●	●	●	◐[3]
mDNS announcement	●	●	●	●	●	●	●	●
SESSION AND INFORMATION								
list / panel info	●	●	●	●	●	●	●	●
_name, _platform, _panelType	●	●	●	●	●	●	●	○
_support flags	●	●	●	●	●	●	◐[4]	○

[1] Blue Pill Platform: '[' arrays work only with exactly one message; others dropped. [2] Link IO: UniSketch Panel only. [3] UniSketch (legacy): Server mode only; values <=200 disable it.

Feature	Blue Pill Platform	Link IO	Stream Deck	X-keys	Touch	HIDs	Raw Panel Dummies	UniSketch (legacy)
map	●	●	●	●	●	●	●	●
PanelTopology?	●	●	●	●	●	●	●	●
ping / ack / nack	●	●	●	●	●	●	●	●
state (state feedback)	○	◐[5]	○	○	○	○	○	○
Connections?	●	●	●	●	●	●	●	●
RunTimeStats?	●	●	●	●	●	●	●	●
NetworkConfig? / SetNetworkConfig	●	○	–	–	–	–	●	○
Registers (Mem, Flag, Shift, State)	○	○	–	–	◐[6]	–	●	◐[7]
FEEDBACK								
HWC# LED state	●	●	●	●	●	◐[8]	●	●
Blink patterns (bits 8-11)	●	●	◐[9]	●	◐[10]	○	●	●
GPO output bit	●	●	–	●	–	–	●	●
HWCc# LED color	●	●	●	●	●	–	●	●
Several HWCs per command (xx,yy)	●	○	●	●	●	◐[11]	●	○
HWCt# display text	●	●	●	–	●	–	●	◐[12]
Text scrolling	○	○	○	–	○	–	○	○
HWCg# mono bitmaps	●	●	●	–	◐[13]	–	●	◐[14]
HWCgGray# gray bitmaps	●	●	●	–	◐[15]	–	●	◐[16]
HWCgRGB# color bitmaps	●	●	●	–	◐[17]	–	●	◐[18]
HWCx# extended values	●	●	●	–	●	–	◐[19]	●
HWCs# individual LEDs	●	○	○	–	○	–	○	●
HWCj# jog wheel	◐[20]	–	–	–	–	–	○	○
Processors	●	○	●	●	●	–	●	○
PANEL SETTINGS								
Clear / ClearLEDs / ClearDisplays	●	●	●	●	●	○	●	●
PanelBrightness=x,y	●	●	●	○	○	–	●	●
PanelBrightness screen value	○	–	○	–	○	–	○	–
DimmedGain	●	●	●	●	○	–	●	●

[4] Raw Panel Dummies: Registers flag missing; added on branch announce-registers-flag. [5] Link IO: Not on UniSketch Panel (disabled on purpose). [6] Touch: Only the TAB state register, which selects a tab. [7] UniSketch (legacy): Set and reported; no Registers? query.

[8] HIDs: HID devices: not supported (states ignored; keyboard lock LEDs not driven); BMD demands: Zoom Demand LED (HWC 8) on/off only; Focus Demand has no outputs. [9] Stream Deck: Approximated, not the exact pattern table. [10] Touch: Approximated, not the exact pattern table. [11] HIDs: HID devices: not supported (states ignored); BMD demands: supported. [12] UniSketch (legacy): 64-char lines, 24-char title, 16 slots per loop. [13] Touch: x,y offset ignored; image is centred. [14] UniSketch (legacy): 8 image slots; x,y offset broken for mono images. [15] Touch: x,y offset ignored; image is centred. [16] UniSketch (legacy): One 12 KB buffer, one image per loop. [17] Touch: x,y offset ignored; image is centred. [18] UniSketch (legacy): One 12 KB buffer, one image per loop. [19] Raw Panel Dummies: No jog wheel type. [20] Blue Pill Platform: JogWheelV2 only; target position only in volume modes.

Feature	Blue Pill Platform	Link IO	Stream Deck	X-keys	Touch	HIDs	Raw Panel Dummies	UniSketch (legacy)
SleepTimer / SleepTimer?	●	●	●	○	○	○	●	●
SleepMode / SleepScreenSaver	●	●	●	○	○	–	●	●
WakeUp!	●	●	●	○	○	○	●	●
Reboot	●	●	●	●	●	●	●	●
EVENTS FROM THE PANEL								
Down / Up with edges	●	●	●	●	●	●	●	●
Abs	●	●	–	●	●	●	●	●
Speed	●	●	–	●	●	●	●	●
Enc	●	●	●	●	●	●	●	●
Abs:x,y / Speed:dx,dy	○	–	–	○	○	○	○	–
Text:	○	–	○	–	–	–	○	–
Raw ADC values	●	–	–	–	–	–	–	–
TOUCHUI								
SetTouchUI and related commands	○	–	–	–	–	–	○	–
HWCo# overlays	○	–	–	–	–	–	○	–
HWCd# runtime domains	○	–	–	–	–	–	○	–
SERVICE								
PublishSystemStat / LoadCPU	●	–	–	–	–	–	–	–
Environmental health	◐[21]	–	–	–	–	–	●	–
BurninProfile?	●	–	●	–	–	–	●	–
Calibration profiles	●	–	–	–	–	–	●	–

[21] Blue Pill Platform: RunMode NORMAL hard-coded; no Simulate.

LEGACY								
BSY / RDY flow control	–	–	–	–	–	–	–	●
HWCmsg# change notification	–	–	–	–	–	–	–	●
encoderPressMode	–	–	–	–	–	–	–	●
Webserver=	–	–	–	–	–	–	–	●

Sources

Implementation	Source inspected
Blue Pill Platform	ibeam-hardware, edbdfdd, 2026-10-05, v1.0.8-pre7 (origin/master); rawpanel-lib submodule 2b78811 (2026-10-02)
Link IO	standalone-firmware, 0d6f8b2 2026-08-25 (branch NV3006-Display, contains origin/master 04c40a2)
Stream Deck	xpanel-streamdeck, 6a75656 2026-10-21 (branch app-ui, no upstream; uncommitted go.mod bumps rawpanel-lib to v1.7.1, committed v1.5.8)
X-keys	xpanel-xkeys, 93efa77 2026-10-21 (branch app-ui = origin/app-ui); rawpanel-lib v1.5.0, rawpanel-server v1.0.7
Touch	xpanel-touch, 633b1cb 2026-10-27 (local branch feature/edit-ui, with uncommitted changes); rawpanel-lib v1.5.0, rawpanel-server v1.0.7
HIDs: HID devices	xpanel-hids, dbe41a3 2026-10-24 (local branch main); rawpanel-lib v1.3.4, rawpanel-server v1.0.5
HIDs: BMD demands	xpanel-bmd-demands, 886a181 2026-10-24 (local branch main, with uncommitted changes); rawpanel-lib v1.5.3, rawpanel-server v1.0.7
Raw Panel Dummies	raw-panel-dummies, 90c4388 2026-02-05 (branch Moving-to-wails); rawpanel-lib v1.5.6, rawpanel-server v1.0.7
UniSketch (legacy)	unisketch, 1c682001 2026-10-02 (origin/master; local checkout 3 behind)

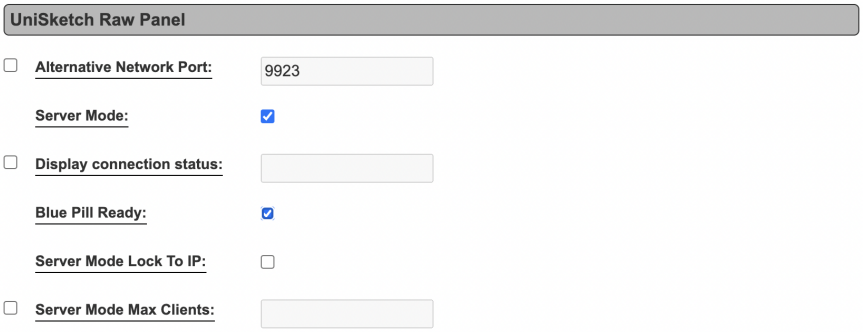
A UniSketch Panels

Raw Panel began as the *UniSketch TCP Client* device core, made to connect UniSketch panels to each other. On UniSketch it is still a device core that you add to a panel's configuration, and it can do a few things that other implementations do not. This appendix covers what you need to know when you work with UniSketch panels.

A.1 Configuration

A UniSketch panel only sends events from components that have the action *Tie to Remote HWC* in its configuration. Raw Panel configurations from SKAARHOJ apply it to every component. Applying it to only some components gives you a partial Raw Panel; the map tells your system which components it gets. Adding the device core twice lets one panel serve two systems.

The device core options set the mode of operation:



Device core options for UniSketch TCP Client (Raw Panel).

- **Server mode**: required for most integrations, including Blue Pill and Reactor. Consider *Lock to IP* and *Max Clients* to prevent unintended connections.
- **Blue Pill Ready**: makes pushed encoders behave like buttons (`Down`/`Up` with edge 16) and their LED rings like button LEDs, as other implementations do. Recommended.
- **Alternative network port**: default 9923.

The legacy option string sets the same options: `D0:0=9930;D0:1=1` sets device core 0 to port 9930 (index 0) in server mode (index 1).

A.2 Blue Pill Direct Mode

For quick tests, any UniSketch panel can run a Raw Panel server regardless of its configuration. Press the key in the lower left corner twice while the color animation runs during boot; a blue and white LED sweep confirms the change, and the panel restarts. Repeat to switch back. The configuration is not affected. You can also type `TakeTheBluePill` in the serial monitor. In this mode the panel uses DHCP and listens on port 9923 in server mode with Blue Pill Ready enabled. For permanent installations, load a proper Raw Panel configuration instead.

A.3 Client mode

In client mode the panel connects to a server at the IP address in its configuration, on port 9923. Everything is ASCII, each line ending with `<NL>`.

1. The panel connects and sends `list`.
2. The server answers `<NL>ActivePanel=1<NL>`. The panel now starts sending events. Send text and graphics only after this, preferably after the panel has sent its map.
3. The panel sends `ping` about every 3 seconds when nothing else is sent. The server must answer, typically with `ack`, or the panel reconnects.
4. The panel sends `list` again about every 60 seconds. The server may resend its feedback in case something was lost.
5. If the panel sends `BSY`, the server pauses until `RDY` or `ping` arrives.

Client mode handshake

←	`list`	panel connected to your server
→	`ActivePanel=1`	preceded and followed by a newline
←	`map=1:1`	
←	`map=2:2`	
→	`HWC#1=36`	send feedback
←	`ping`	
→	`ack`	

In server mode, UniSketch panels need no handshake.

A.4 Registers and change notifications

When two UniSketch panels are connected through Raw Panel, one running *UniSketch TCP Client* and the other *TCP Server*, they can also share registers: shift levels, states, memories and flags. The actions *Shift Level*, *State*, *Memory* and *Flags* of the TCP Client device core then act on the registers of the remote panel, not the local one. The inbound commands `Mem`, `Flag#`, `Shift` and `State` set them, and `_state:` and `_shift:` report changes. A known use is linking two ETH-GPI Link boxes over a network.

The action *Change Notification* sends `HWCmsg#xx=CN:a,b,c,d,e` whenever one of its five parameters changes. Parameters can be a fixed value 0–31, a memory (A–H), a shift or state register, a group of eight flags, or a timer (1/8 to 64 seconds). Use it to tell your system that the panel's context changed, for example that a shift level was selected and new display content is needed.

B Changelog

Date	Changes
2026-10	Individual LEDs in LED bars (`HWCs`). State feedback (`state`, `HWCstate`, support flag `StateFeedback`). Text scrolling in rawpanel-lib and the proto. Support flag `Segments`. Document rewritten; protocol versions V1/V2 replaced by support flags and an implementation support matrix.
2026-08	Haptic jog wheel configuration (`HWCj`, flag `JogConfig`). Text events (`HWC#xx=Text:`) and vector events (`Abs:x,y`, `Speed:dx,dy`). Screen backlight as third value of `PanelBrightness`.
2026-07	TouchUI: widget configuration for touch-screen panels, overlays (`HWCo`), runtime domains (`HWCd`), off-screen bit in the map.
2026-02	Scrolling text (field 14 of `HWCt`, flag `TextScroll`).
2025-06	WebP in GfxConv. Icon processor. UniText with four lines and alignment.
2024-06	Network settings (`NetworkConfig?`, `SetNetworkConfig`).
2023-10	Multi-dimensional inputs (`dim=` in topology).
2023-07	Processors (meters, image conversion, UTF-8 text). Output type `rb`.
2023-02	Input type `pi` and extended type `jogcfg` for jog wheels.
2022-10	Panel type and support flags in the reply to `list`. Calibration profiles. Environmental health. JSON on outbound and full JSON messages inbound. Registers on Blue Pill panels.
2022-08	`render` key, `stepList` and `max` parameters in topologies.
2022-07	Protocol auto-detection documented. Raw Panel Explorer.
2020-05	`ClearLEDs`, `ClearDisplays`, `PanelBrightness`, `Webserver`. Faster server mode.
2020-01	Text rendering: more fonts, proportional fonts, ASCII 32-127, 24-character strings. Image sizes other than 64×32. Up to 255 HWCs. 32-bit values. Icons.
2019-08	Server mode. `nack` for unknown commands. `Clear`, `Reboot`. Extended type 5 for faders.
2019-05	Panel topology.
2019-01	Encoder press. Text formats 9-11. Software version. Sleep commands. Extended values (`HWCx`). Automatic reports of state, shift and sleep.
2017-09	First version.